DINESPOT

A FULL-STACK RESTAURANT DISCOVERY AND TABLE RESERVATION PLATFORM

A PROJECT REPORT

Submitted by

HEMANT V S (310624104358)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY, CHENNAI

NOVEMBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report "**DINESPOT: A FULL-STACK RESTAURANT DISCOVERY AND TABLE RESERVATION PLATFORM**" is the bonafide work of "**HEMANT V S (310624104358)**" who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. S. Cloudin
Professor and Head
Department of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai – 600089

SUPERVISOR
[Supervisor Name]
[Designation]
Department of Computer Science and Engineering
Easwari Engineering College
Ramapuram, Chennai – 600089

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce Examination

This is to certify that the Viva-voce Examination of the project work entitled "**DINESPOT: A FULL-STACK RESTAURANT DISCOVERY AND TABLE RESERVATION PLATFORM**" submitted by **HEMANT V S (310624104358)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering was conducted on _____ in the Department of Computer Science and Engineering, Easwari Engineering College, Ramapuram, Chennai.

Internal Examiner	**External Examiner**
Date:	Date:

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal, Dr. P. Deiva Sundari, and the Management of Easwari Engineering College for providing me with the opportunity and facilities to carry out this project work.

I convey my heartfelt thanks to Dr. S. Cloudin, Head of the Department of Computer Science and Engineering, for his continuous encouragement and support.

I am extremely thankful to my project supervisor, [Supervisor Name], Department of Computer Science and Engineering, for the valuable guidance, suggestions and motivation given throughout the completion of this project.

I also extend my gratitude to all the faculty members and staff of the Department of Computer Science and Engineering for their support.

Finally, I thank my parents and friends for their constant encouragement and help during the course of this project.

HEMANT V S

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Abstract	i
2	List of Figures	ii
3	List of Tables	iii
4	List of Abbreviations	iv
5	**Chapter 1 – Introduction**	**1**
	1.1 Project Overview	1
	1.2 Problem Statement	2
	1.3 Objectives of the Project	2
	1.4 Scope of the Project	3
6	**Chapter 2 – Existing System and Proposed System**	**5**
	2.1 Existing System	5
	2.2 Limitations of the Existing System	5
	2.3 Proposed Solution	6
	2.4 Advantages of the Proposed System	6
7	**Chapter 3 – System Analysis and Design**	**8**
	3.1 Hardware and Software Requirements	8
	3.1.1 Software Requirements	8
	3.1.2 Hardware Requirements	9
	3.2 System Architecture	9
	3.3 Module Description	11
	3.3.1 Restaurant Discovery Module	11
	3.3.2 Search and Area Filter Module	12
	3.3.3 Session Module	12
	3.3.4 Reservation and Menu Pre-ordering Module	12
	3.3.5 Booking History Module	12
	3.3.6 Admin Reservation Management Module	12
	3.3.7 Database Design	13
	3.4 UML Diagrams / Data Flow Diagram	14
	3.4.1 Use Case Diagram	14
	3.4.2 Data Flow Diagrams	15
	3.4.3 Entity Relationship Diagram	16
	3.4.4 Booking Status Lifecycle	17

S. No.	Contents	Page No.
	3.4.5 Activity Diagram	17
	3.5 Algorithm / Flowchart	18
8	**Chapter 4 – Implementation**	**21**
	4.1 Implementation Details	21
	4.1.1 Database Initialisation	21
	4.1.2 Idempotent Seeding	22
	4.2 Module Implementation	22
	4.2.1 Restaurant Search API	22
	4.2.2 Booking Validation	23
	4.2.3 Transactional Table Allocation	23
	4.2.4 Session Management with React Context	24
	4.2.5 Reservation Page	25
	4.2.6 Admin Status Update	25
	4.3 Tools and Technologies Used	26
9	**Chapter 5 – Results and Testing**	**28**
	5.1 Screenshots and Outputs	28
	5.2 Testing and Validation	35
	5.3 Results and Discussion	37
10	**Chapter 6 – Conclusion and Future Enhancement**	**39**
	6.1 Conclusion	39
	6.2 Future Enhancements	39
11	References / Bibliography	41
12	Appendices (Source Code)	43

ABSTRACT

Reserving a restaurant table is still largely handled by telephone calls and handwritten registers, which leads to double bookings, poor use of seating, late communication of menu preferences and no visible status for the customer. DineSpot addresses this problem through a full-stack Restaurant Discovery and Table Reservation Platform built with React, Node.js, Express and SQLite. Customers browse a rated collection of seventeen restaurants across seven Chennai localities, search by area or name or use quick area chips, and reserve a table for a chosen date, time and party size while pre-ordering dishes from vegetarian and non-vegetarian menus. The Express REST API validates each request and runs a smallest-fit table-allocation algorithm inside a single SQLite transaction: among the tables of the restaurant that are large enough and not already reserved for the same date and time, the one with the least capacity is assigned. This prevents double booking and keeps larger tables free for larger groups. Every booking records the assigned table, the pre-ordered items and a fixed booking fee of ₹50, and starts in the pending state. Hotel staff manage requests on an admin dashboard by accepting, rejecting or completing them, and customers follow the status on their profile page. The frontend uses React 19, Vite, React Router, Framer Motion and Lucide React, while the backend uses Express 5 and better-sqlite3 with automatic schema creation and seeding. API and interface tests confirmed correct searching, validation, conflict-free allocation and status management, and also identified the limitations of the current version, namely client-side sign-in, an unprotected admin page and a simulated payment. These are proposed as future enhancements.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System Architecture of DineSpot	10
3.2	Use Case Diagram of DineSpot	15
3.3	Level 0 Data Flow Diagram	16
3.4	Level 1 Data Flow Diagram	16
3.5	Entity Relationship Diagram	17
3.6	Booking Status Lifecycle	17
3.7	Activity Diagram for Table Reservation	18
3.8	Flowchart of the Table Allocation Algorithm	20
5.1	Home Page with Search Bar and Area Chips	28
5.2	Restaurant Collection	29
5.3	Area Search Result for Adyar	29
5.4	Area with No Restaurants	30
5.5	Sign In Page	30
5.6	Member Registration Form	31
5.7	Reservation Page	31
5.8	Validation Message When No Dish Is Selected	32
5.9	Menu Items Selected	32
5.10	Reservation Confirmed	33
5.11	Booking History on the Profile Page	33
5.12	Admin Reservation Dashboard	34
5.13	Dashboard After Accepting a Booking	34

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	DineSpot at a glance	1
1.2	Objectives and how they are met in DineSpot	3
2.1	Comparison of the existing process and DineSpot	6
3.1	Software requirements	8
3.2	Recommended hardware for development	9
3.3	Layers of the system	9
3.4	REST API endpoints	10
3.5	Frontend pages and routes	11
3.6	Summary of modules	13
3.7	Database tables	13
3.8	Structure of the bookings table	14
4.1	Main project folders	21
4.2	Libraries and frameworks	26
4.3	Development tools	26
4.4	Run commands	27
5.1	Test cases and results	35
5.2	Summary of test results	37
6.1	Roadmap of future work	40

LIST OF ABBREVIATIONS

S. No.	ABBREVIATION	EXPANSION
1	API	Application Programming Interface
2	CORS	Cross-Origin Resource Sharing
3	CSS	Cascading Style Sheets
4	DFD	Data Flow Diagram
5	ER	Entity Relationship
6	FK	Foreign Key
7	HTML	HyperText Markup Language
8	HTTP	Hypertext Transfer Protocol
9	JSON	JavaScript Object Notation
10	JSX	JavaScript XML
11	JWT	JSON Web Token
12	PK	Primary Key
13	REST	Representational State Transfer
14	SMS	Short Message Service
15	SPA	Single-Page Application
16	SQL	Structured Query Language
17	UI	User Interface
18	UML	Unified Modeling Language
19	URL	Uniform Resource Locator
20	WAL	Write-Ahead Logging

CHAPTER 1
INTRODUCTION

1.1 PROJECT OVERVIEW

DineSpot is a full-stack web application for discovering restaurants and reserving a table in advance. A customer opens a single page, sees a curated collection of restaurants in Chennai, narrows the list to a neighbourhood such as Ramapuram, Anna Nagar or Adyar, and reserves a table for a chosen date, time and number of guests. While reserving, the customer can also pre-order dishes from a vegetarian or non-vegetarian menu, so that the restaurant knows what to prepare before the guests arrive. A hotel administrator then reviews each request on a reservation dashboard and accepts, rejects or completes it.

The frontend is a single-page application written in React 19 and built with Vite. React Router DOM provides client-side navigation between the Home, Login, Reservation (Order Summary), Profile and Admin pages, Framer Motion supplies page and card animations, and Lucide React supplies the icon set. The backend is a Node.js server written with Express 5 that exposes a small REST API under the `/api` prefix. Persistent data is stored in a file-based SQLite database (`dinespot.db`) that is accessed through the synchronous `better-sqlite3` driver. The database is created and seeded automatically when the server starts, so the project can be run on a fresh machine without any manual database setup.

The feature that distinguishes DineSpot from a simple enquiry form is its **table-allocation logic**. Every restaurant has a set of physical tables with fixed seating capacities. When a booking request arrives, the server looks for tables that are not already reserved for the same restaurant, date and time, chooses the smallest table that can seat the party, and records the booking inside a single database transaction. A request that cannot be satisfied is rejected with a clear message instead of creating a double booking. Table 1.1 summarises the project at a glance.

Table 1.1 DineSpot at a glance

Item	Description
Project title	DineSpot – A Full-Stack Restaurant Discovery and Table Reservation Platform
Application type	Single-page web application with a REST API backend
Users	Customer (guest or signed-in) and Admin (hotel staff)

Item	Description
Frontend	React 19, Vite 8, React Router DOM 7, Framer Motion, Lucide React, CSS3
Backend	Node.js with Express 5, CORS middleware
Database	SQLite through better-sqlite3 (WAL journal mode, foreign keys enabled)
Seed data	17 restaurants in 7 Chennai areas with 295 tables of 2, 4 or 6 seats
Core workflow	Discover → reserve table and pre-order menu → admin approval → booking history

1.2 PROBLEM STATEMENT

Reserving a table at a restaurant is still largely handled by telephone calls or by walking in and waiting. A customer who telephones has to repeat the date, time and party size, often receives no written confirmation, and cannot easily compare several restaurants in the same area before deciding. During busy hours, calls go unanswered and walk-in customers wait without knowing whether a table will become free.

Restaurants face the opposite side of the same problem. When reservations are noted manually in a register, two staff members can promise the same table for the same slot, and the kitchen learns what the guests want to eat only after they are seated. Large parties may be given a small table, while a couple may occupy a six-seat table that a later family needed. Without a central record, the staff cannot see at a glance which requests are pending, which are confirmed and which have been completed.

The problem addressed by this project is therefore to provide a single system in which customers can discover restaurants by locality, reserve a suitably sized table for a specific slot without the risk of double booking, communicate their menu choices in advance, and follow the status of their requests, while restaurant staff manage all incoming reservations from one dashboard.

1.3 OBJECTIVES OF THE PROJECT

The main objective of DineSpot is to design and implement a working full-stack reservation platform using React on the client side, Express on the server side and SQLite as the relational data store. The specific objectives, and the way each is met in the implementation, are listed in Table 1.2.

Table 1.2 Objectives and how they are met in DineSpot

Objective	How it is met in DineSpot
Restaurant discovery	Home page lists all restaurants sorted by rating; cards show cuisine, area, hours, price band and rating
Locality-based search	Search bar and area chips call GET /api/restaurants?q=… which matches area or restaurant name
Conflict-free reservation	Server allocates the smallest free table for the slot inside a database transaction
Menu pre-ordering	Customer selects dishes from vegetarian and non-vegetarian lists; items are stored with the booking
Input validation	Client checks required fields and menu selection; server validates e-mail, date format and guest range
Booking history	Profile page lists the signed-in user's bookings with table number, menu and status
Reservation management	Admin dashboard lists all bookings and lets staff accept, reject or mark them completed
Zero-setup persistence	SQLite schema is created and restaurant data is seeded automatically on server start

1.4 SCOPE OF THE PROJECT

DineSpot is intended for diners in Chennai and for the staff of the participating restaurants. The present version is an academic implementation that runs on a local machine, with a deployment configuration prepared for Vercel. Its scope is defined below.

Inside the scope of this version

- Listing restaurants with name, area, address, cuisine, rating, price band, opening hours and image.
- Searching restaurants by area or name and filtering through quick-select area chips.
- Client-side sign-in, member registration form and guest access, with the session kept in the browser.
- Reserving a table for a date, time and party size of up to ten guests from the interface.
- Pre-ordering vegetarian and non-vegetarian dishes as part of the reservation.
- Automatic allocation of the smallest available table and prevention of double booking for a slot.
- Recording a fixed booking fee of ₹50 against each reservation.

- Viewing personal booking history and managing all reservations from an admin dashboard.

Outside the scope of this version

- Server-side user accounts, password verification and token-based authentication.
- Integration with a real payment gateway; the booking fee is recorded but no money is collected.
- Role-based protection of the admin dashboard.
- Cancellation or modification of bookings by the customer.
- E-mail, SMS or push notifications when a booking status changes.
- Native Android or iOS applications.

CHAPTER 2
EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 EXISTING SYSTEM

In the existing process, a customer who wants to dine at a restaurant either calls the restaurant to reserve a table or arrives in person and waits. The details of a telephone reservation are written in a register or remembered by the person who took the call. Information about restaurants in a locality is gathered from word of mouth, review websites or general listing applications, which describe the restaurant but are often not connected to the restaurant's own table records.

Menu preferences are communicated only after the guests are seated. Larger groups sometimes call ahead to explain their order, but this information is again recorded informally. When the restaurant wants to know how many reservations are pending or confirmed for an evening, a staff member has to read through the register by hand.

2.2 LIMITATIONS OF THE EXISTING SYSTEM

- **Risk of double booking.** Without a single record of which table is reserved for which slot, two customers can be promised the same table.
- **Poor table utilisation.** Tables are assigned by judgement rather than by capacity, so small parties may occupy large tables.
- **No written confirmation.** A customer who books by phone receives no record of the date, time, party size or table.
- **Fragmented discovery.** Customers must use separate sources to find restaurants in an area and to reserve a table.
- **Late menu information.** The kitchen learns the order only at the table, which increases waiting time during peak hours.
- **No status tracking.** Neither the customer nor the staff can see whether a request is pending, accepted, rejected or completed.
- **Manual effort.** Staff spend time answering calls and maintaining registers instead of serving guests.

2.3 PROPOSED SOLUTION

The proposed system, DineSpot, replaces the informal process with a web application backed by a relational database. Restaurants, their physical tables and every booking are stored in SQLite tables that are linked by foreign keys. The customer discovers restaurants on the home page, filters them by area, and opens a reservation page that combines the booking form with a menu selector. When the customer confirms, the backend validates the request, checks which tables are already reserved for the same restaurant, date and time, and assigns the smallest free table that can seat the party. The booking is stored with status `pending` and the customer immediately sees a confirmation summary.

Hotel staff use the admin dashboard to see every booking together with the requested menu, assigned table and fee status. Each pending request can be accepted or rejected, and an accepted request can later be marked as completed. The customer sees the updated status on the Profile page. Table 2.1 compares the existing process with DineSpot.

Table 2.1 Comparison of the existing process and DineSpot

Aspect	Existing process	DineSpot
Discovery	Word of mouth, separate apps	Rated restaurant cards with area search and chips
Booking channel	Telephone or walk-in	Web reservation form
Table assignment	Manual, by judgement	Smallest free table that fits the party, chosen by the server
Double booking	Possible	Prevented for the same restaurant, date and time
Menu preferences	Told at the table	Pre-ordered with the reservation
Confirmation	Verbal	On-screen summary and booking history
Request management	Paper register	Admin dashboard with accept, reject and complete actions

2.4 ADVANTAGES OF THE PROPOSED SYSTEM

- **Conflict-free allocation:** table selection and booking insertion run inside one SQLite transaction, so a table already reserved for a slot is never assigned again for that slot.
- **Better use of seating:** ordering candidate tables by ascending capacity places a party of two at a two-seat table and keeps larger tables free for larger groups.

- **Faster service:** dishes chosen in advance are visible to staff on the dashboard before the guests arrive.
- **Transparency:** customers see the assigned table number, fee and current status of every booking.
- **Centralised management:** all reservations are listed in one place with clear status badges.
- **Simple deployment:** SQLite needs no separate database server, and the schema and data are created automatically.
- **Responsive interface:** React renders updates without page reloads, and the API responds with JSON that the interface displays directly.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 HARDWARE AND SOFTWARE REQUIREMENTS

DineSpot is a JavaScript application end to end. The same runtime, Node.js, runs the development server for the frontend and the Express server for the backend, and SQLite is embedded in the backend process through the `better-sqlite3` native module. Table 3.1 lists the software used, with the versions installed in the project, and Table 3.2 gives a recommended development configuration.

3.1.1 Software Requirements

Table 3.1 Software requirements

Component	Software / Version	Purpose
Operating system	Windows 10/11, Linux or macOS	Development and execution
Runtime	Node.js (v24.12.0 used during development)	Runs Vite and the Express server
Package manager	npm	Installs frontend and backend dependencies
Frontend library	React 19.3.0, React DOM	Component-based user interface
Build tool	Vite 8.3.1 with @vitejs/plugin-react	Development server, proxy and production build
Routing	React Router DOM 7.18.4	Client-side page navigation
Animation / icons	Framer Motion 13.4.6, Lucide React 1.48.0	Transitions and icons
Backend framework	Express 5.2.1, cors	REST API and cross-origin support
Database	SQLite via better-sqlite3 13.0.3	Embedded relational storage
Code quality	oxlint 1.81	Static analysis of frontend code
Browser	Any modern browser (Google Chrome used for testing)	Running the client application
Editor	Visual Studio Code	Source code editing

3.1.2 Hardware Requirements

Table 3.2 Recommended hardware for development

Component	Recommended specification
Processor	Dual-core 64-bit processor or better
Memory (RAM)	4 GB minimum; 8 GB recommended when the editor, browser and two Node.js processes run together
Storage	About 500 MB free for the project, node_modules and the SQLite database file
Network	Required for installing packages and for loading restaurant images and web fonts
Display	1366 × 768 or higher resolution

3.2 SYSTEM ARCHITECTURE

DineSpot follows a three-tier client–server architecture, shown in Figure 3.1. The **presentation tier** is the React single-page application that runs in the browser. It renders pages, keeps the signed-in user in a React context, and stores the session and the selected restaurant in the browser's localStorage. The **application tier** is the Express server listening on port 5000; it applies middleware, routes each request to a handler, validates input and runs the table-allocation transaction. The **data tier** is the SQLite database file `backend/database/dinespot.db`, which holds four tables.

During development the Vite server on port 5173 serves the frontend and forwards every request that starts with `/api` to `http://localhost:5000` through its proxy setting, so the browser sees a single origin. The frontend reads an optional `VITE_API_URL` variable so that the API base address can be changed for deployment. A `vercel.json` file in the project root describes a deployment in which the backend and frontend run as separate Vercel services and `/api/*` requests are rewritten to the backend. Table 3.3 summarises the responsibilities of each tier.

Table 3.3 Layers of the system

Layer	Technology	Responsibility
Presentation	React, React Router, Framer Motion, CSS	Render pages, collect input, client-side validation, show results
Client state	React Context API, localStorage	Hold signed-in user and selected restaurant across pages

Layer	Technology	Responsibility
Communication	fetch API, JSON, Vite proxy	Send HTTP requests to /api and parse JSON responses
Application	Node.js, Express 5, cors	Routing, validation, table allocation, error handling
Data access	better-sqlite3 prepared statements	Parameterised SQL queries and transactions
Persistence	SQLite (WAL mode)	Store restaurants, tables, bookings and users

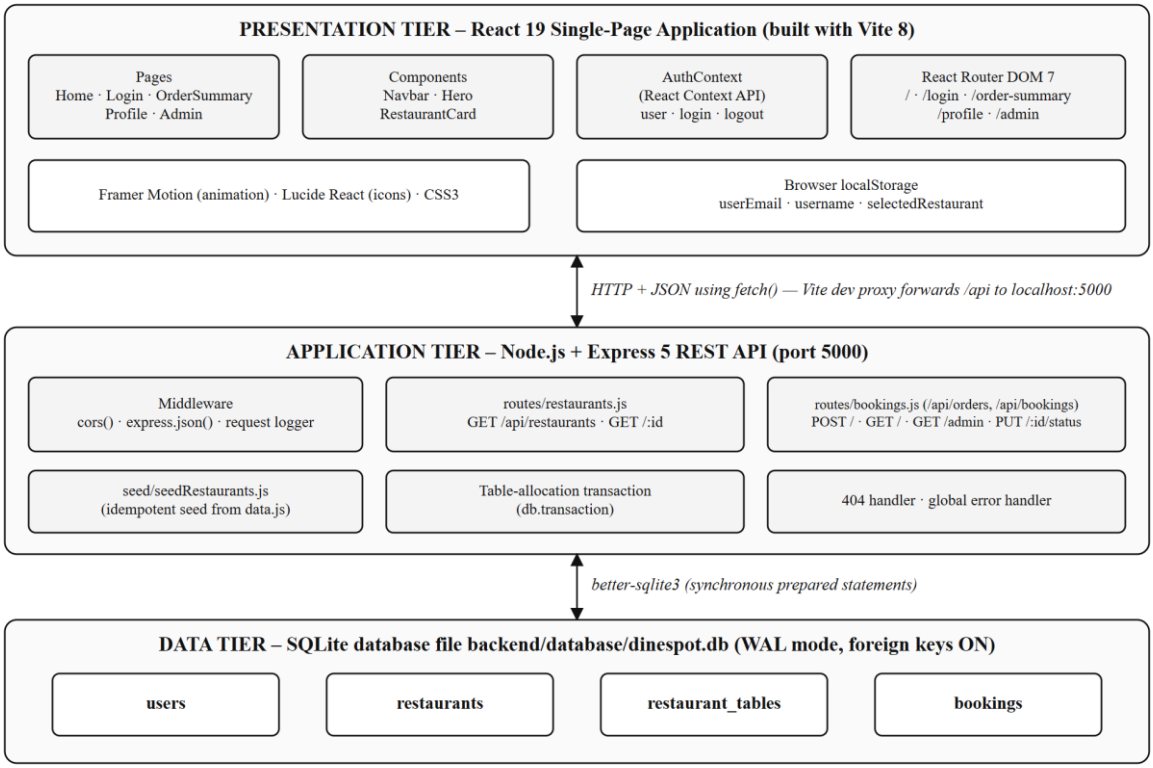


Figure 3.1 System Architecture of DineSpot

The REST endpoints exposed by the backend are listed in Table 3.4. The booking router is mounted twice, at `/api/orders` (used by the frontend) and at `/api/bookings` (a REST-style alias), so both prefixes reach the same handlers.

Table 3.4 REST API endpoints

Method	Endpoint	Description
GET	/api/restaurants	All restaurants ordered by rating (highest first)
GET	/api/restaurants?q=text	Restaurants whose area or name contains the text (case-insensitive)

Method	Endpoint	Description
GET	/api/restaurants?region=text	Restaurants whose area contains the text
GET	/api/restaurants/:id	One restaurant; 404 if the id does not exist
POST	/api/orders	Validate request, allocate a table and create a booking
GET	/api/orders?email=…	Bookings of one user with restaurant name and table number
GET	/api/orders/admin	All bookings for the admin dashboard
PUT	/api/orders/:id/status	Change the status of a booking
—	any other path	404 JSON response “Route not found”

3.3 MODULE DESCRIPTION

The application is organised into six functional modules. The frontend pages and the routes that reach them are listed in Table 3.5, and each module is described below.

Table 3.5 Frontend pages and routes

Route	Page component	Purpose
/	Home (with Hero and RestaurantCard)	Search, area chips and restaurant collection
/login	Login	Sign in, member registration form and guest access
/order-summary	OrderSummary	Reservation form, menu selection and confirmation
/profile	Profile	Booking history of the signed-in user and sign out
/admin	Admin	Reservation dashboard for hotel staff

3.3.1 Restaurant Discovery Module

This module consists of the Home page, the Hero component and the RestaurantCard component. On load, the Home page requests `GET /api/restaurants` and displays every restaurant in a three-column grid under the heading “The Culinary Collection”. Each card shows the image, rating badge, name, price band, cuisine, area and opening hours, together with a **Reserve Table** button.

3.3.2 Search and Area Filter Module

The Hero component contains a search bar and nine area chips (Ramapuram, Anna Nagar, T. Nagar, Adyar, Velachery, Porur, Mylapore, Koyambedu and Saidapet). Submitting the search form or clicking a chip calls `GET /api/restaurants?q=…`. The result header shows the number of matches and the selected area, and a **Show All** button restores the full list. If no restaurant matches, a message such as "No culinary destinations in Koyambedu" is displayed. When the request fails because the server cannot be reached, the page filters the already loaded list in the browser instead.

3.3.3 Session Module

The Login page offers a sign-in form (e-mail and password), a member registration form (full name, e-mail and password) and a **Continue as Guest** option. The `AuthContext` provider stores the user's e-mail and display name in React state and in localStorage, and exposes `login` and `logout` functions to all components. The Navbar reads this context to show either a **Sign In** link or the user's name, an **Admin** link and a **Sign Out** button. This module is a client-side session only; the password is required by the form but is not sent to or verified by the server.

3.3.4 Reservation and Menu Pre-ordering Module

The OrderSummary page shows the selected restaurant's image and address on the left and the reservation form on the right. The customer selects a date, a time and a party size from one to ten guests, and chooses dishes from two tabs containing sixteen vegetarian and sixteen non-vegetarian items. The page shows a fixed booking fee of ₹50. On confirmation, the page sends the request to `POST /api/orders` and, on success, replaces the form with a "Reservation Confirmed" summary listing the date, time, guests and pre-ordered dishes.

3.3.5 Booking History Module

The Profile page greets the signed-in user and fetches `GET /api/orders?email=…`. Each booking is shown as a card containing the restaurant name, a coloured status badge, date, time, number of guests, assigned table, area, the list of pre-ordered dishes, the booking fee with its payment status and the date on which the booking was made.

3.3.6 Admin Reservation Management Module

The Admin page, titled "Reservation Dashboard", fetches `GET /api/orders/admin` and lists every booking with the customer's e-mail. Pending bookings show **Accept** and **Reject**

buttons; accepted bookings show a **Mark Completed** button. Each button sends `PUT /api/orders/:id/status` and updates the card in place when the server confirms the change. Table 3.6 summarises all modules.

Table 3.6 Summary of modules

Module	Main files	API used
Restaurant discovery	Home.jsx, RestaurantCard.jsx	GET /api/restaurants
Search and area filter	Hero.jsx, Home.jsx	GET /api/restaurants?q=
Session	Login.jsx, AuthContext.jsx, Navbar.jsx	None (browser only)
Reservation and pre-ordering	OrderSummary.jsx, routes/bookings.js	POST /api/orders
Booking history	Profile.jsx	GET /api/orders?email=
Admin management	Admin.jsx	GET /api/orders/admin, PUT /api/orders/:id/status

3.3.7 Database Design

The relational schema is defined in `backend/database/database.js` and is created with `CREATE TABLE IF NOT EXISTS` statements every time the server starts. Table 3.7 describes the four tables and Table 3.8 gives the structure of the central `bookings` table.

Table 3.7 Database tables

Table	Key columns	Purpose
restaurants	id (TEXT, PK), name, region, address, cuisine, rating, lat, lng	Catalogue of restaurants (17 seeded rows)
restaurant_tables	id (PK), restaurant_id (FK, ON DELETE CASCADE), table_number, capacity	Physical tables of each restaurant (295 seeded rows)
bookings	id (PK), user_email, restaurant_id (FK), table_id (FK), booking_date, booking_time	Every reservation request
users	id (PK), name, email (UNIQUE)	Defined for future server-side accounts; not used by the current code

Table 3.8 Structure of the bookings table

Column	Type / Default	Description
id	INTEGER PK AUTOINCREMENT	Booking number
user_email	TEXT NOT NULL	E-mail of the customer who booked
restaurant_id	TEXT NOT NULL, FK	Restaurant being booked
table_id	INTEGER, FK	Table allocated by the server
booking_date / booking_time	TEXT NOT NULL	Slot in YYYY-MM-DD and HH:MM form
guests	INTEGER NOT NULL	Party size (1 to 20 accepted by the API)
menu_items	TEXT	Pre-ordered dishes stored as a JSON array
location	TEXT	Area of the restaurant at the time of booking
fee_amount	REAL DEFAULT 50.0	Booking fee in rupees
payment_status	TEXT DEFAULT 'pending'	Set to 'paid' when the booking is created
status	TEXT DEFAULT 'pending'	pending, accepted, rejected or completed
created_at	TEXT DEFAULT datetime('now')	Time the booking was recorded

3.4 UML DIAGRAMS / DATA FLOW DIAGRAM

3.4.1 Use Case Diagram

Figure 3.2 shows the use case diagram. The **Customer** actor can browse the collection, search by area or name, filter using an area chip, sign in or register, and continue as a guest. A **Registered Customer** is a specialisation of Customer who has an active session and can reserve a table, view booking history and sign out. Reserving a table includes pre-ordering menu items and confirming the ₹50 booking fee, and confirmation includes the system's allocation of the smallest free table. The **Admin** actor views all reservations and accepts, rejects or completes them.

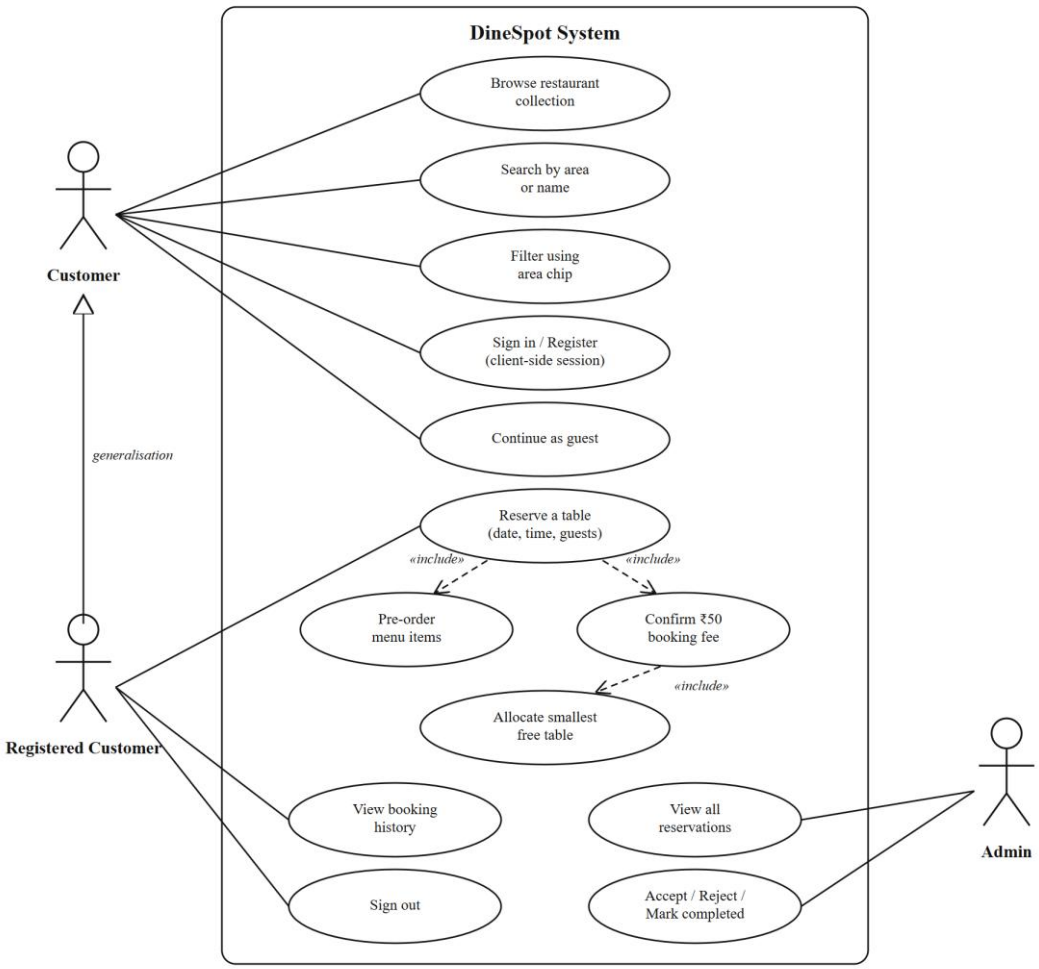


Figure 3.2 Use Case Diagram of DineSpot

3.4.2 Data Flow Diagrams

The Level 0 data flow diagram in Figure 3.3 treats DineSpot as a single process exchanging data with the two external entities and the database. Figure 3.4 refines it into five processes: searching restaurants, managing the session, creating a booking, viewing booking history and managing reservations. It also shows the four data stores, including the browser's localStorage, which holds the session.

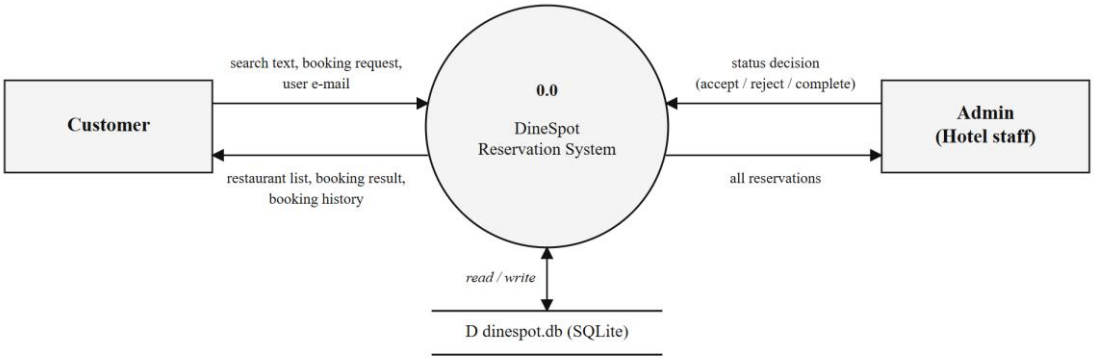


Figure 3.3 Level 0 Data Flow Diagram

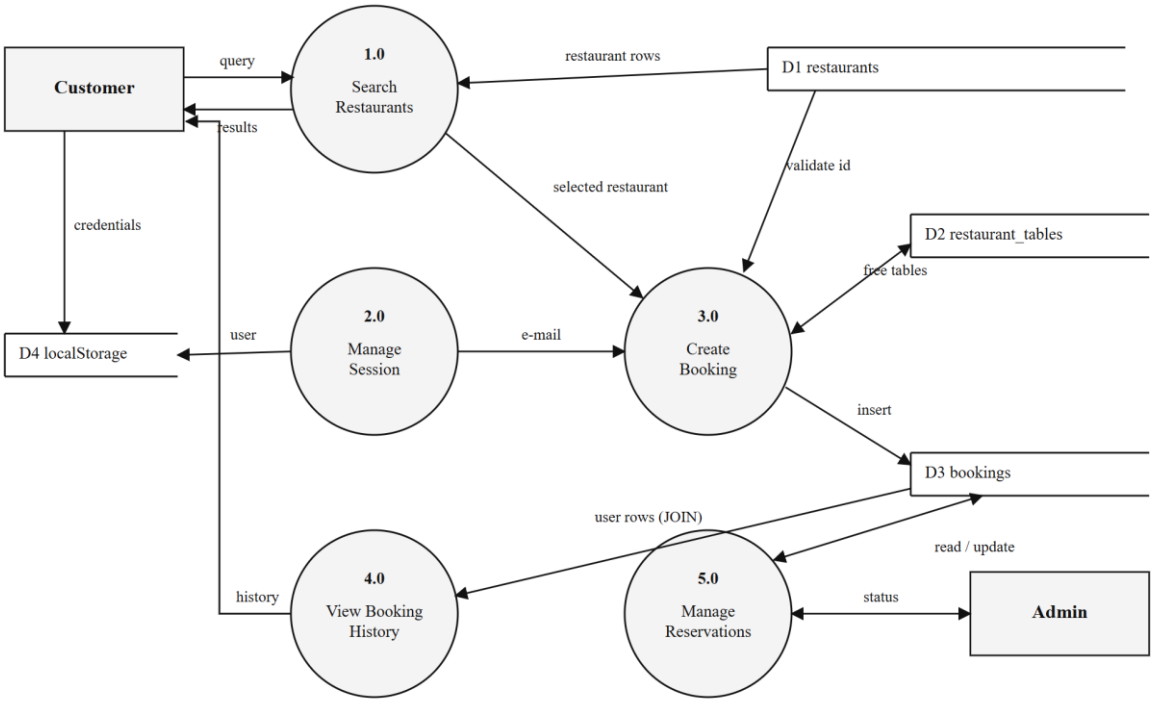


Figure 3.4 Level 1 Data Flow Diagram

3.4.3 Entity Relationship Diagram

Figure 3.5 shows the database entities and their relationships. A restaurant has many tables and receives many bookings, and each booking is assigned to exactly one table. The `users` table is related to bookings only logically, through the e-mail address, because the current version does not store accounts on the server.

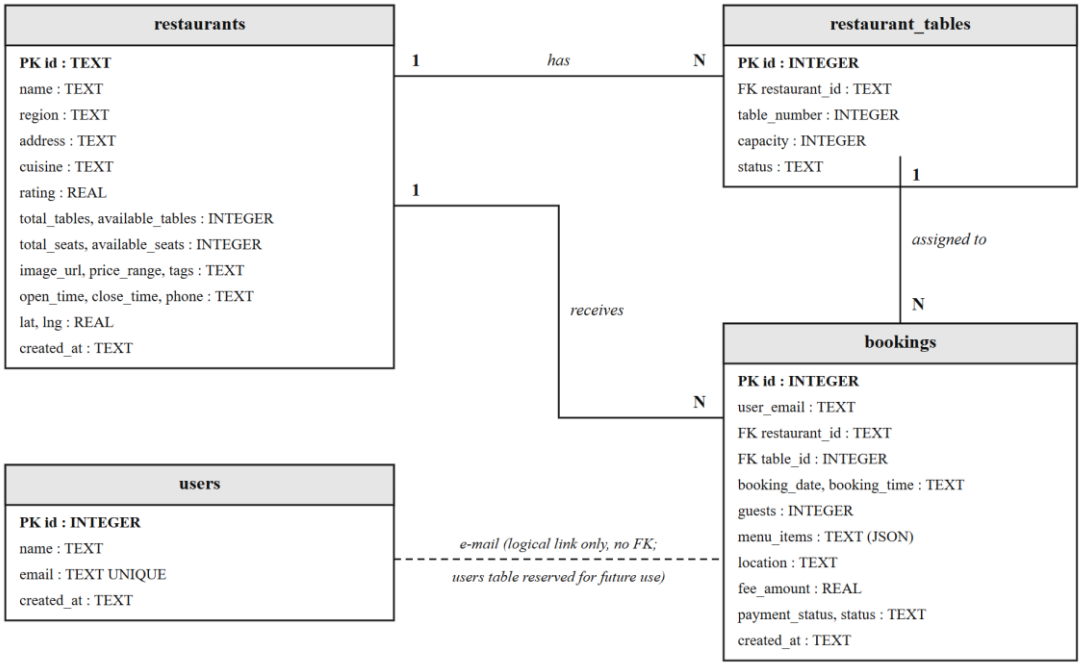


Figure 3.5 Entity Relationship Diagram

3.4.4 Booking Status Lifecycle

Every booking is created with status `pending`. The admin can move it to `accepted` or `rejected`, and an accepted booking can be moved to `completed`, as shown in Figure 3.6. The allocation query ignores bookings whose status is `cancelled`, but no screen in the current version sets that status.

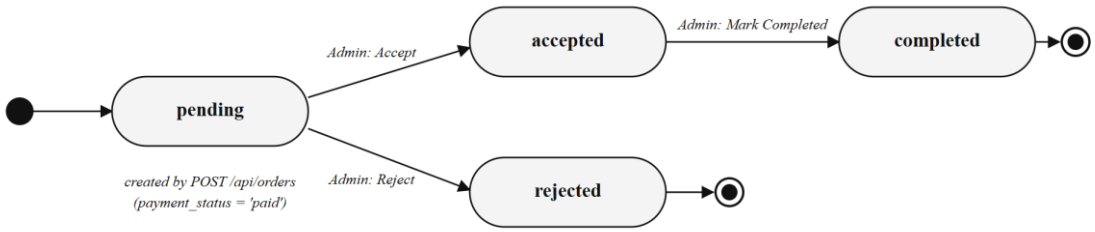


Figure 3.6 Booking Status Lifecycle

3.4.5 Activity Diagram

The activity diagram in Figure 3.7 follows a customer from opening the home page to receiving a confirmation. Two decision points come from the client (whether the user is signed in, and whether the form and menu selection are complete) and one comes from the server (whether a table could be allocated).

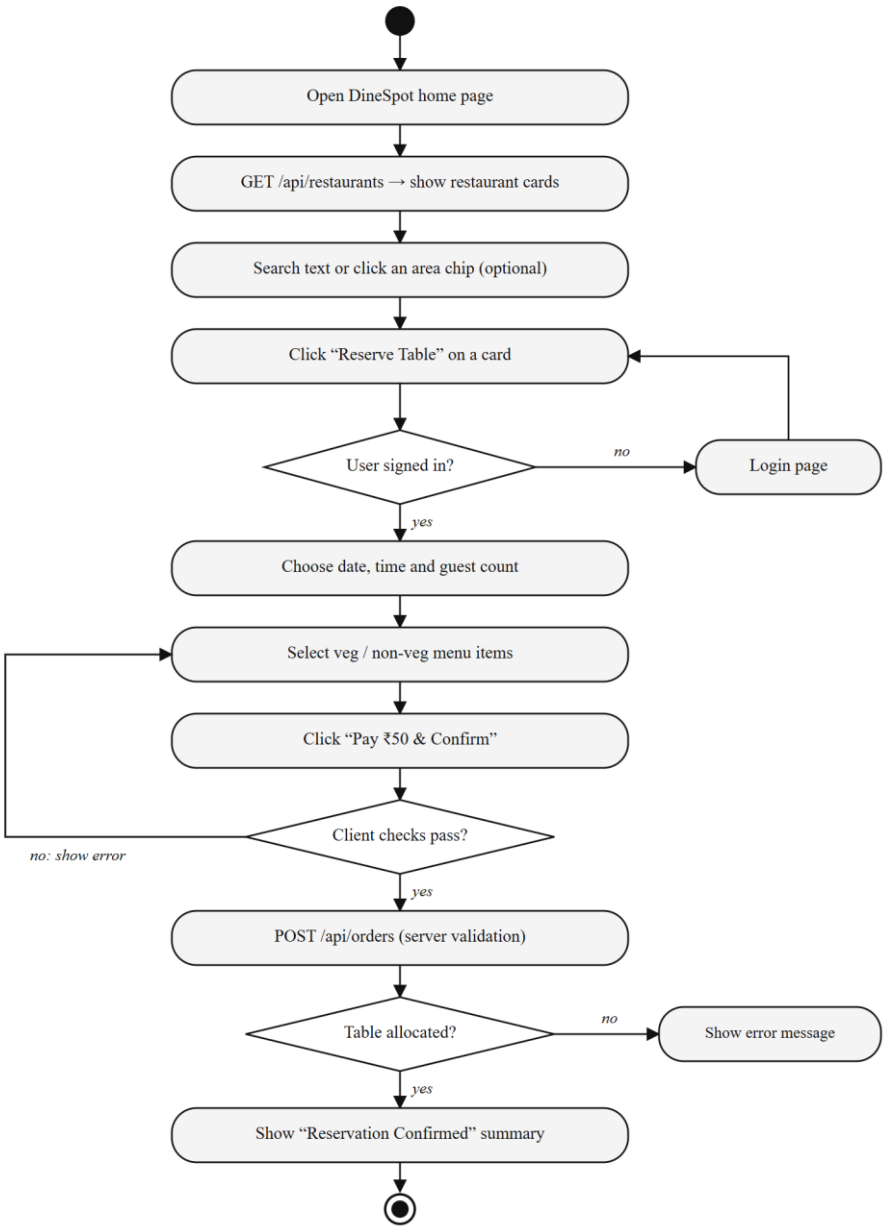


Figure 3.7 Activity Diagram for Table Reservation

3.5 ALGORITHM / FLOWCHART

The central algorithm of DineSpot is the **smallest-fit table allocation** performed by `POST /api/orders`. It is a best-fit strategy: among all tables that are large enough and free for the requested slot, it picks the one with the least capacity. The steps are given below and the flowchart is shown in Figure 3.8.

Algorithm: AllocateTable(userEmail, restaurantId, date, time, people, menuItems)

Step 1: If userEmail, orderDetails, restaurantId, date or time is missing, return HTTP 400 with the name of the missing field.

Step 2: Convert people to an integer guests; if it is not a number or is outside 1 to 20, return HTTP 400.

Step 3: If date does not match the pattern YYYY-MM-DD, return HTTP 400.

Step 4: Look up the restaurant by id; if it does not exist, return HTTP 404.

Step 5: Begin a database transaction.

Step 6: bookedIds ← table_id of every booking for the same restaurant, date and time whose status is not cancelled.

Step 7: Select one table of the restaurant with capacity ≥ guests and id not in bookedIds, ordered by capacity ascending.

Step 8: If no table is found, return HTTP 409 “No table available for the selected time slot and guest count”.

Step 9: Insert the booking with the chosen table, the menu items as JSON, fee 50.0, payment status paid and status pending.

Step 10: Commit the transaction and return the booking id, table number and table capacity.

If the restaurant has n tables and b bookings already exist for the slot, Step 6 reads b rows and Step 7 scans at most n tables, so the work per request is proportional to n + b. Because `better-sqlite3` runs the whole transaction synchronously inside the Node.js event loop, no other request can run between Step 6 and Step 9. This prevents two requests from the same server process from taking the same table.

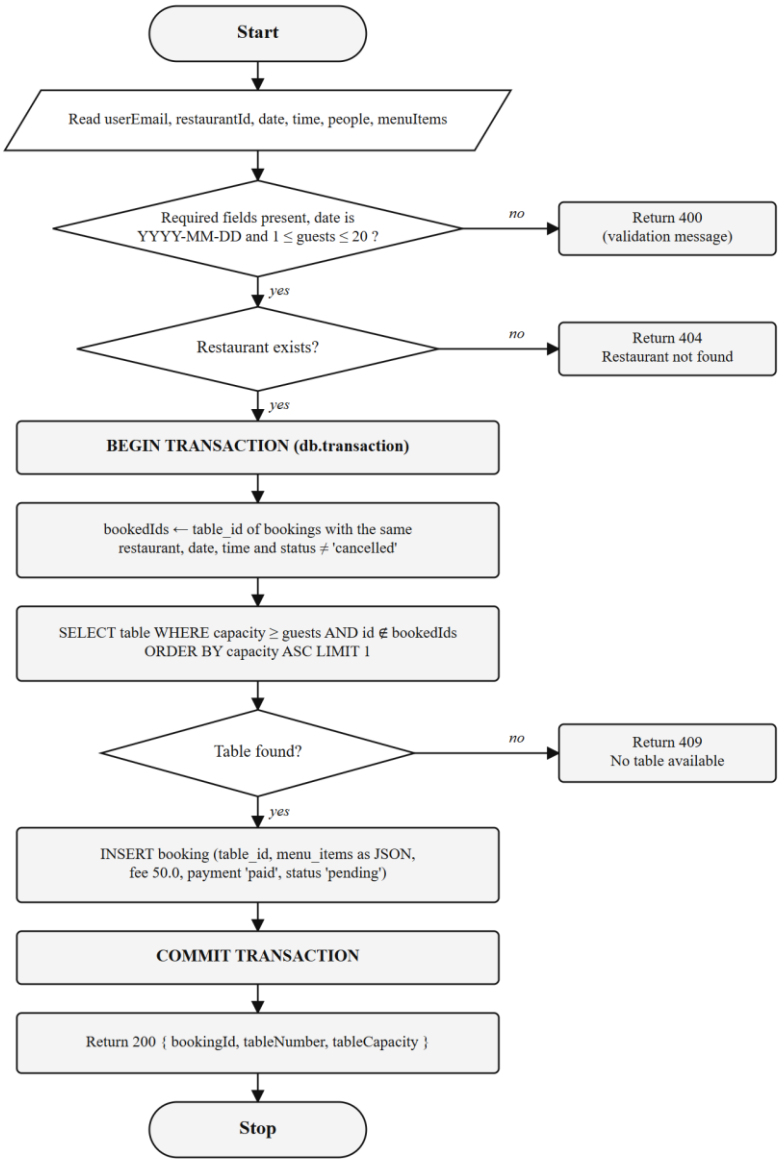


Figure 3.8 Flowchart of the Table Allocation Algorithm

CHAPTER 4
IMPLEMENTATION

4.1 IMPLEMENTATION DETAILS

The project is a monorepo with separate `frontend` and `backend` folders, each with its own `package.json`. The backend uses CommonJS modules and the frontend uses ES modules compiled by Vite. Table 4.1 shows the main folders.

Table 4.1 Main project folders

Folder / File	Contents
backend/server.js	Express application, middleware, route mounting, 404 and error handlers
backend/database/	database.js (connection and schema) and the dinespot.db file
backend/routes/	restaurants.js and bookings.js route handlers
backend/seed/	seedRestaurants.js, which loads data.js into SQLite
backend/data.js	Seed records for 17 restaurants with coordinates and opening hours
frontend/src/pages/	Home, Login, OrderSummary, Profile and Admin pages with their CSS
frontend/src/components/	Navbar, Hero, RestaurantCard and MapComponent
frontend/src/context/	AuthContext.jsx (session provider)
frontend/vite.config.js	React plugin and /api proxy to port 5000
vercel.json	Two-service deployment configuration with /api rewrite
legacy/	Earlier static HTML, CSS and JavaScript prototype; not used by the running application

The `MapComponent` file contains a Leaflet map that places a marker at each restaurant's coordinates. It is present in the source tree but is not imported by any page in the current version, so the map is not displayed in the running application.

4.1.1 Database Initialisation

Listing 4.1 shows how the database connection is opened. Write-Ahead Logging allows readers to continue while a write is in progress, and the `foreign_keys` pragma makes SQLite enforce the references declared in the schema.

Listing 4.1 ***Opening the SQLite database (backend/database/database.js)***

```
const Database = require('better-sqlite3');
const db = new Database(path.join(dbDir, 'dinespot.db'));

// Enable WAL mode for better concurrent read performance
db.pragma('journal_mode = WAL');
// Enforce foreign keys
db.pragma('foreign_keys = ON');

db.exec(`
  CREATE TABLE IF NOT EXISTS restaurant_tables (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    restaurant_id TEXT NOT NULL REFERENCES restaurants(id) ON DELETE CASCADE,
    table_number  INTEGER NOT NULL,
    capacity      INTEGER NOT NULL,
    status        TEXT DEFAULT 'available'
  );
  ...
`);
```

4.1.2 Idempotent Seeding

On every start, `seedRestaurants()` inserts the records from `data.js` with `INSERT OR IGNORE`, so restaurants that already exist are skipped. For each newly inserted restaurant, it creates as many tables as the restaurant's `totalTables` value, cycling through the capacities 2, 2, 4, 4, 4, 6 and 6, as shown in Listing 4.2.

Listing 4.2 ***Generating tables for a new restaurant (seed/seedRestaurants.js)***

```
if (result.changes > 0) {
  seeded++;
  // Generate sensible tables for each restaurant
  const tableSizes = [2, 2, 4, 4, 4, 6, 6];
  for (let i = 0; i < r.totalTables; i++) {
    insertTable.run({
      restaurantId: r.id,
      tableNumber: i + 1,
      capacity: tableSizes[i % tableSizes.length]
    });
  }
}
```

4.2 MODULE IMPLEMENTATION

4.2.1 Restaurant Search API

The restaurant router builds a parameterised `LIKE` query depending on the query string. Converting both sides to lower case makes the search case-insensitive, and the `%` wildcards allow partial matches such as “anna” for Anna Nagar. A helper, `parseRestaurant`, converts the stored JSON `tags` string back into an array and adds camel-case field names expected by the frontend.

Listing 4.3 ***Search by area or name (routes/restaurants.js)***

```
router.get('/', (req, res) => {
  const { region, q } = req.query;
  let stmt, rows;
  if (region) {
    stmt = db.prepare(`SELECT * FROM restaurants WHERE LOWER(region) LIKE LOWER(?)  ORDER
    BY rating DESC`);
    rows = stmt.all(`%${region}%`);
  } else if (q) {
    stmt = db.prepare(`SELECT * FROM restaurants WHERE LOWER(region) LIKE LOWER(?) OR
    LOWER(name) LIKE LOWER(?) ORDER BY rating DESC`);
    rows = stmt.all(`%${q}%`, `%${q}%`);
  } else {
    rows = db.prepare('SELECT * FROM restaurants ORDER BY rating DESC').all();
  }
  res.json(rows.map(parseRestaurant));
});
```

4.2.2 Booking Validation

Before any database work, the booking handler checks the request body and returns a 400 response that names the problem, as shown in Listing 4.4. The same message is displayed to the user by the reservation page.

Listing 4.4 ***Validation of a booking request (routes/bookings.js)***

```
if (!userEmail) return res.status(400).json({ success: false, message: 'userEmail is
    required' });
if (!orderDetails) return res.status(400).json({ success: false, message: 'orderDetails is
    required' });
const { restaurantId, date, time, people, menuItems } = orderDetails;
if (!restaurantId) return res.status(400).json({ success: false, message: 'restaurantId is
    required' });
if (!date) return res.status(400).json({ success: false, message: 'date is required' });
if (!time) return res.status(400).json({ success: false, message: 'time is required' });


const guests = parseInt(people, 10);
if (isNaN(guests) || guests < 1 || guests > 20) {
  return res.status(400).json({ success: false, message: 'guests must be between 1 and 20'
    });
}
if (!/^\d{4}-\d{2}-\d{2}$/.test(date)) {
  return res.status(400).json({ success: false, message: 'Invalid date format. Use YYYY-MM-
    DD' });
}
```

4.2.3 Transactional Table Allocation

Listing 4.5 is the implementation of the algorithm described in Section 3.5. `db.transaction()` wraps the function so that the reads and the insert are committed together, or rolled back together if an exception occurs. When some tables are already booked, the handler generates one `?` placeholder per booked id so that the ids are passed as bound parameters rather than concatenated into the SQL string.

Listing 4.5 ***Smallest-fit allocation inside a transaction (routes/bookings.js)***

```
const bookingTx = db.transaction(() => {
```

```
  const bookedRows = db.prepare(`
    SELECT table_id FROM bookings
    WHERE restaurant_id = ? AND booking_date = ? AND booking_time = ?
      AND status != 'cancelled'
  `).all(restaurantId, date, time);
  const bookedIds = bookedRows.map(b => b.table_id);

  let availableTable;
  if (bookedIds.length === 0) {
    availableTable = db.prepare(`
      SELECT * FROM restaurant_tables
      WHERE restaurant_id = ? AND capacity >= ?
      ORDER BY capacity ASC LIMIT 1
    `).get(restaurantId, guests);
  } else {
    const placeholders = bookedIds.map(() => '?').join(',');
    availableTable = db.prepare(`
      SELECT * FROM restaurant_tables
      WHERE restaurant_id = ? AND capacity >= ?
        AND id NOT IN (${placeholders})
      ORDER BY capacity ASC LIMIT 1
    `).get(restaurantId, guests, ...bookedIds);
  }
  if (!availableTable) {
    return { success: false, status: 409,
      message: 'No table available for the selected time slot and guest count' };
  }
  const result = db.prepare(`INSERT INTO bookings (...) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
    50.0, 'paid', 'pending')`)
    .run(userEmail, restaurantId, availableTable.id, date, time, guests,
         JSON.stringify(menuItems || []), restaurant.region);
  return { success: true, bookingId: result.lastInsertRowid,
           tableNumber: availableTable.table_number, tableCapacity:
    availableTable.capacity, ... };
});
```

4.2.4 Session Management with React Context

The `AuthProvider` in Listing 4.6 restores the user from localStorage when the application loads and writes to localStorage on sign-in and sign-out. Any component can call `useAuth()` to read the user or to sign in and out.

Listing 4.6 ***Client-side session provider (context/AuthContext.jsx)***

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  useEffect(() => {
    const savedUserEmail = localStorage.getItem('userEmail');
    const savedUserName = localStorage.getItem('username');
    if (savedUserEmail) setUser({ email: savedUserEmail, name: savedUserName });
  }, []);
  const login = (userData) => {
    setUser(userData);
    localStorage.setItem('userEmail', userData.email);
    localStorage.setItem('username', userData.name || userData.email);
  };
  const logout = () => {
    setUser(null);
    localStorage.removeItem('userEmail');
    localStorage.removeItem('username');
  };
  return <AuthContext.Provider value={{ user, login, logout
    }}>{children}</AuthContext.Provider>;
};
```

Because the stored user is restored inside an effect, the first render after a full page load has `user` set to `null`. The Profile and OrderSummary pages redirect to `/login` when they see a null user, so opening `/profile` directly, or refreshing it, sends a signed-in user to the login page. Navigating to the page through the Navbar link works correctly. This behaviour was observed during testing and is recorded in Chapter 5.

4.2.5 Reservation Page

The reservation handler in Listing 4.7 checks that the date, time and party size are filled and that at least one dish is selected, then posts the request. The server's message is shown in an error box if the booking fails, and a separate message is shown if the server cannot be reached.

Listing 4.7 ***Submitting a reservation (pages/OrderSummary.jsx)***

```
const handleBooking = async (e) => {
  e.preventDefault();
  setBookingError('');
  if (!bookingDate || !bookingTime || !bookingPeople) {
    setBookingError('Please complete all reservation details.'); return;
  }
  if (selectedItems.length === 0) {
    setBookingError('Please select at least one menu item.'); return;
  }
  setIsSubmitting(true);
  const orderDetails = { restaurantId: restaurant.id, restaurantName: restaurant.name,
    date: bookingDate, time: bookingTime, people: bookingPeople, menuItems: selectedItems
    };
  try {
    const response = await fetch(`${API}/api/orders`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ userEmail: user.email, orderDetails })
    });
    const data = await response.json();
    if (data.success) setIsSuccess(true);
    else setBookingError(data.message || 'Reservation failed. Please try again.');
  } catch (err) {
    setBookingError('Cannot reach the server. Please ensure the backend is running.');
  } finally {
    setIsSubmitting(false);
  }
};
```

4.2.6 Admin Status Update

On the server, `PUT /api/orders/:id/status` runs `UPDATE bookings SET status = ? WHERE id = ?` and returns 404 when no row changes. On the client, the Admin page updates the matching card in its local state only after the server reports success (Listing 4.8), so the dashboard always reflects the stored status.

Listing 4.8 ***Updating a booking status (pages/Admin.jsx)***

```
const updateStatus = async (id, newStatus) => {
  try {
```

```
    const res = await fetch(`${API}/api/orders/${id}/status`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status: newStatus })
    });
    const data = await res.json();
    if (data.success) {
      setOrders(prev => prev.map(o => o.id === id ? { ...o, status: newStatus } : o));
    } else {
      alert('Failed to update status');
    }
  } catch (err) {
    alert('Error connecting to server');
  }
};
```

4.3 TOOLS AND TECHNOLOGIES USED

Table 4.2 lists the libraries and frameworks used in DineSpot together with the role each plays. Table 4.3 lists the development tools, and Table 4.4 gives the commands used to run the project.

Table 4.2 Libraries and frameworks

Library	Version	Role in DineSpot
React / React DOM	19.3.0	Component model, state and rendering
Vite	8.3.1	Development server with /api proxy, production bundling
React Router DOM	7.18.4	Routes for the five pages
Framer Motion	13.4.6	Entry, scroll and hover animations
Lucide React	1.48.0	Icons for search, calendar, clock, users and status
Leaflet / React Leaflet	1.9.4 / 5.0.0	Installed; used only by the unmounted MapComponent
Express	5.2.1	HTTP server, routing and JSON body parsing
cors	2.8.x	Cross-Origin Resource Sharing headers
better-sqlite3	13.0.3	Synchronous SQLite access, prepared statements, transactions
dotenv	18.0.x	Listed as a dependency; not loaded by the current server code

Table 4.3 Development tools

Tool	Use
Visual Studio Code	Writing and organising source code
Node.js and npm	Running servers and managing packages

Tool	Use
oxlint	Linting the frontend (npm run lint)
Google Chrome	Running and testing the application
curl	Sending test requests to the REST API
Git and GitHub Desktop	Version control

Table 4.4 Run commands

Purpose	Command
Install backend packages	cd backend → npm install
Start backend on port 5000	npm run dev (runs node server.js)
Install frontend packages	cd frontend → npm install
Start frontend on port 5173	npm run dev (runs vite)
Production build of frontend	npm run build
Lint frontend code	npm run lint

CHAPTER 5
RESULTS AND TESTING

5.1 SCREENSHOTS AND OUTPUTS

Figures 5.1 to 5.13 were captured from the running application at a browser width of 1440 pixels, with the Express backend on port 5000 and the Vite development server on port 5173. The bookings shown were created through the interface during testing.

Figure 5.1 shows the landing section. The search bar accepts an area or restaurant name, and the nine chips below it start an area search with one click.

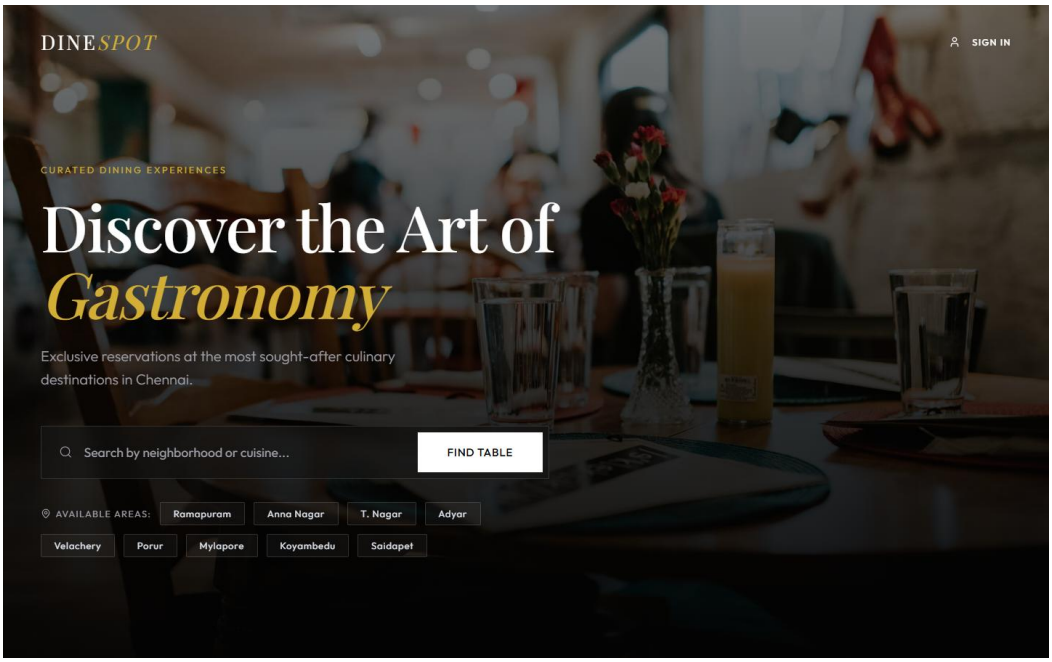


Figure 5.1 Home Page with Search Bar and Area Chips

Figure 5.2 shows the restaurant grid sorted by rating. Each card displays the rating badge, price band, cuisine, area and opening hours.

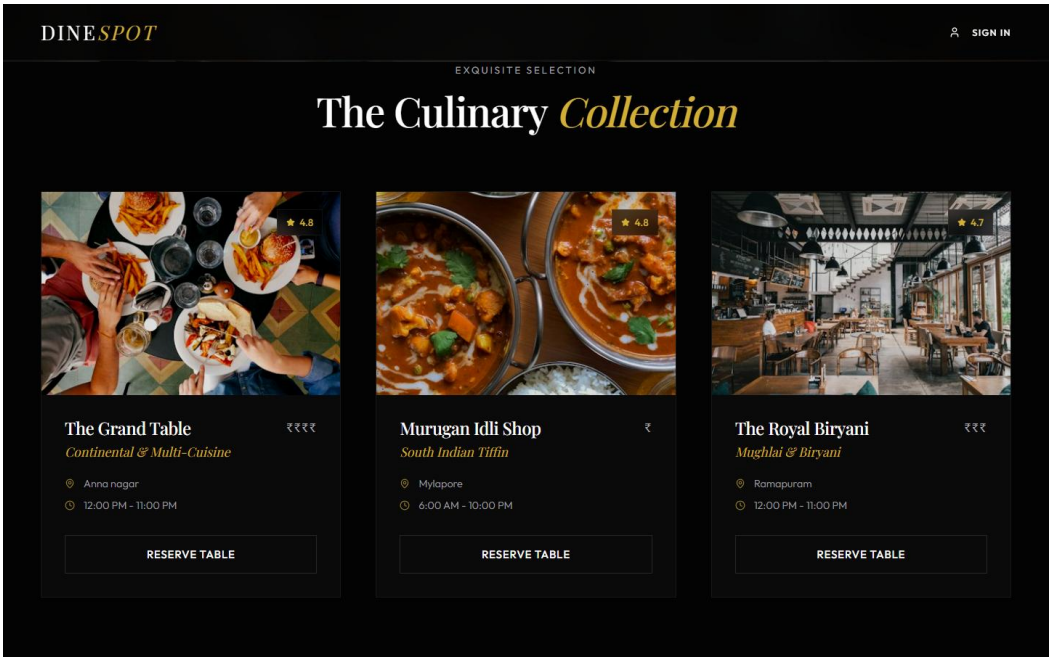


Figure 5.2 Restaurant Collection

Selecting the Adyar chip returns the two restaurants in that area, with a count badge and a Show All button (Figure 5.3).

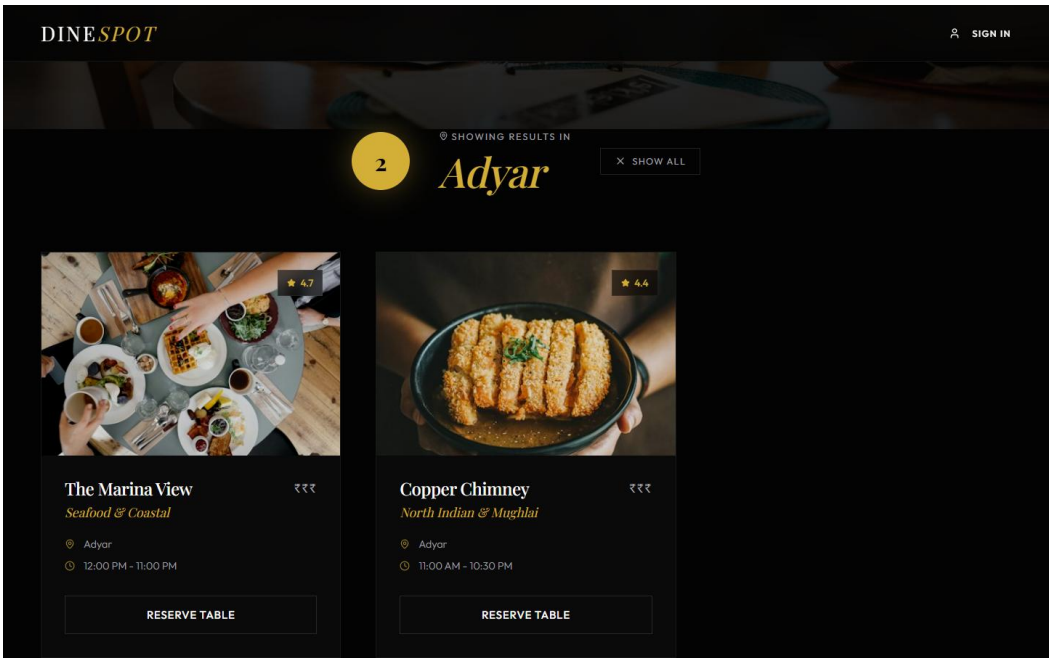


Figure 5.3 Area Search Result for Adyar

Koyambedu appears as a chip but has no restaurants in the seed data, so the page shows the empty-result message in Figure 5.4.

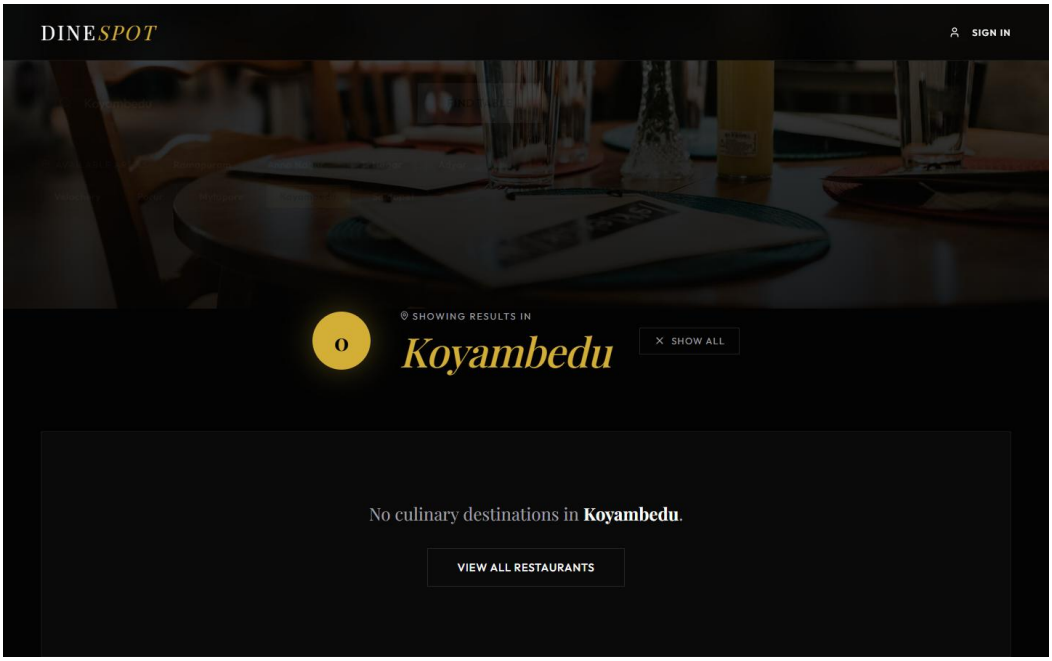


Figure 5.4 Area with No Restaurants

Figure 5.5 shows the sign-in form with e-mail and password fields and the Continue as Guest option.

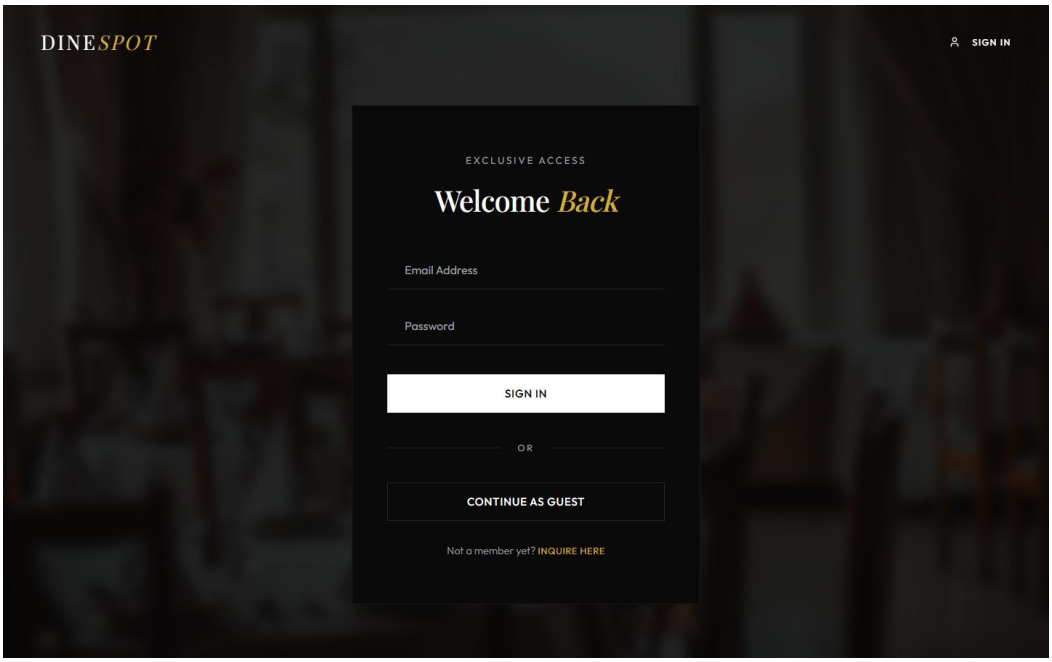


Figure 5.5 Sign In Page

Selecting “Inquire here” switches the same page to the registration form, which also asks for the full name (Figure 5.6).

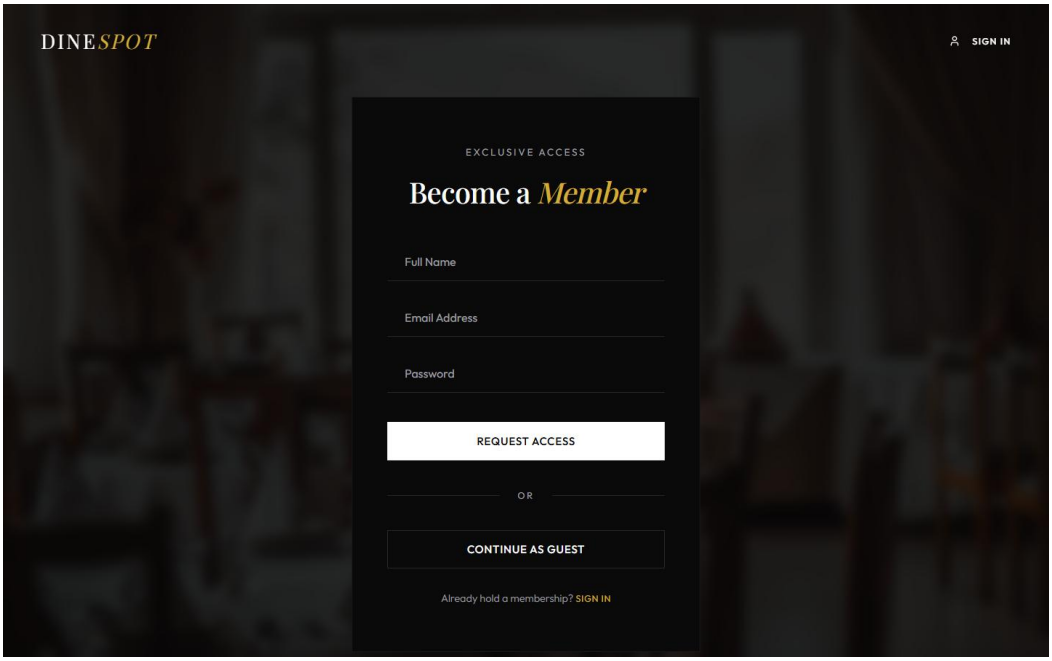


Figure 5.6 Member Registration Form

Figure 5.7 shows the reservation page for Spice Garden, with the venue details and menu on the left and the booking form with the ₹50 fee on the right.

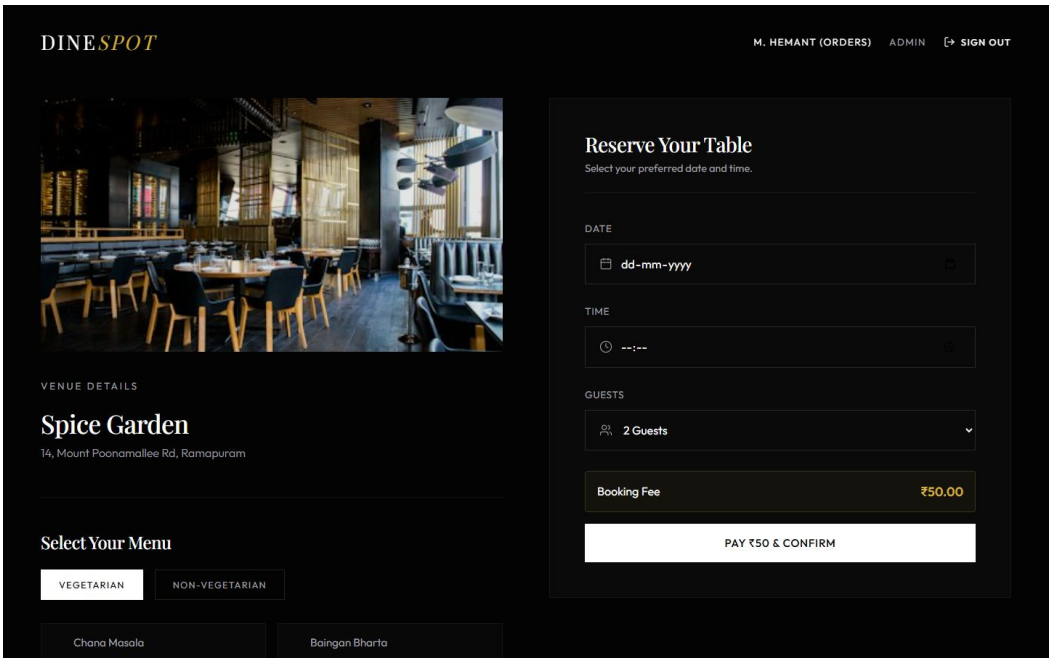


Figure 5.7 Reservation Page

Submitting the form with a date, time and party size but no dish produces the message “Please select at least one menu item.” (Figure 5.8).

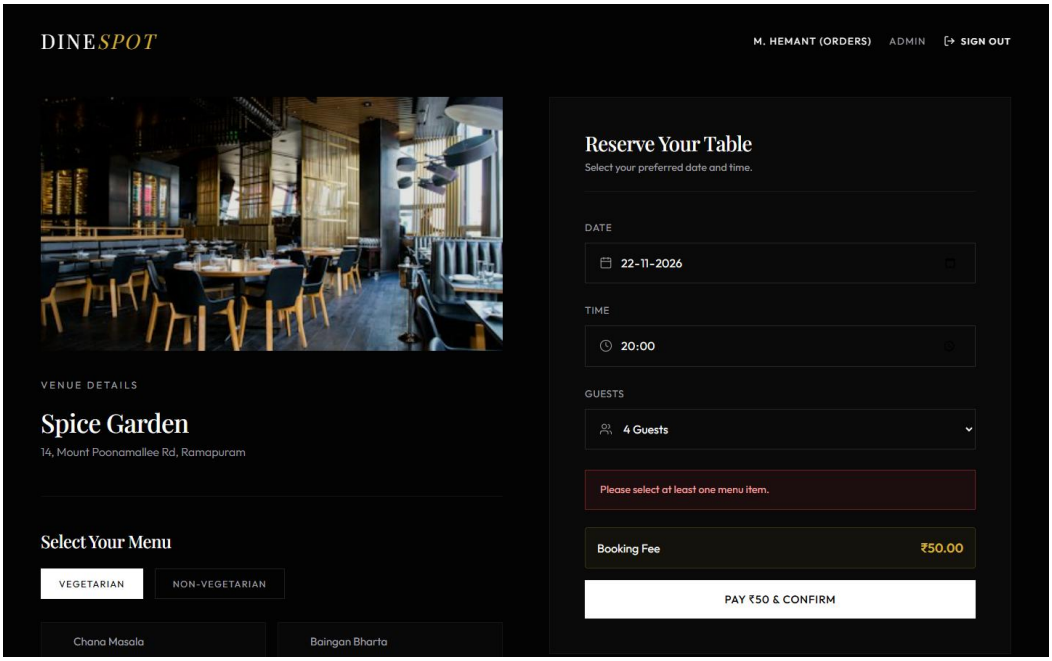


Figure 5.8 Validation Message When No Dish Is Selected

Selected dishes are highlighted and counted below the menu (Figure 5.9). Selections are kept when switching between the vegetarian and non-vegetarian tabs.

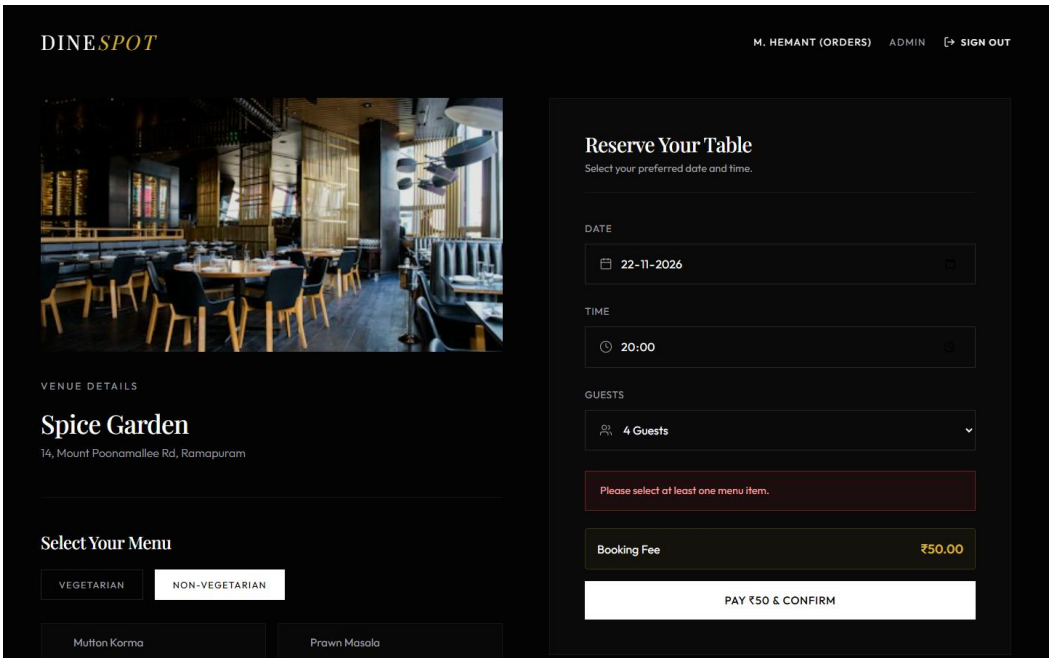


Figure 5.9 Menu Items Selected

After a successful request, the page shows the confirmation in Figure 5.10 with the date, time, guests and pre-ordered dishes. The server assigned Table 3, a four-seat table, to this party of four.

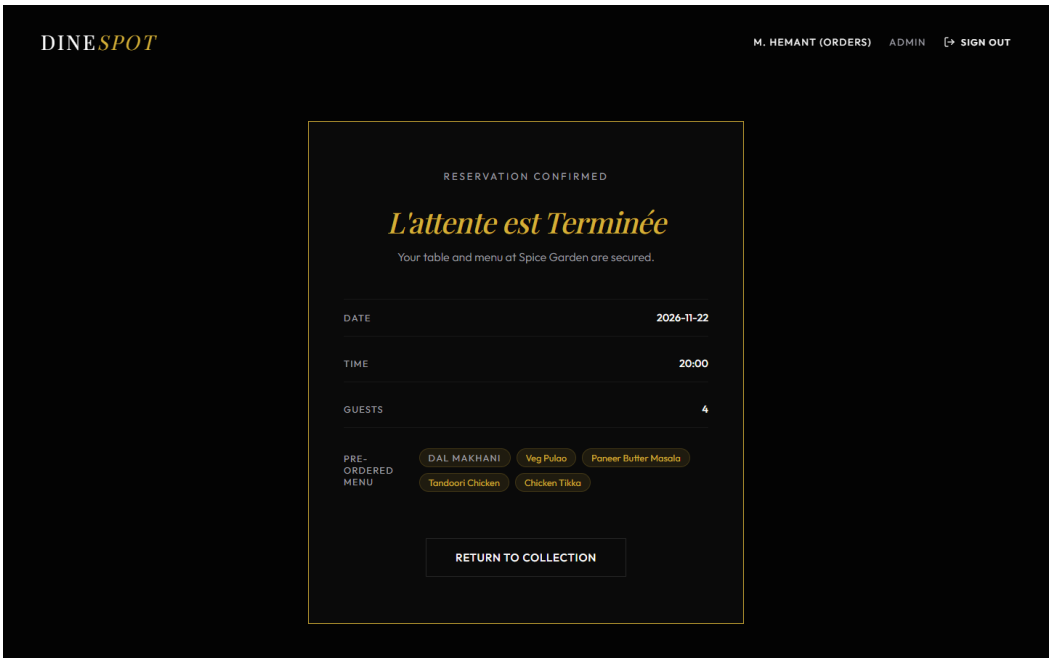


Figure 5.10 Reservation Confirmed

Figure 5.11 shows the Profile page reached through the Navbar link. Each booking shows its status badge, table number, menu and fee.

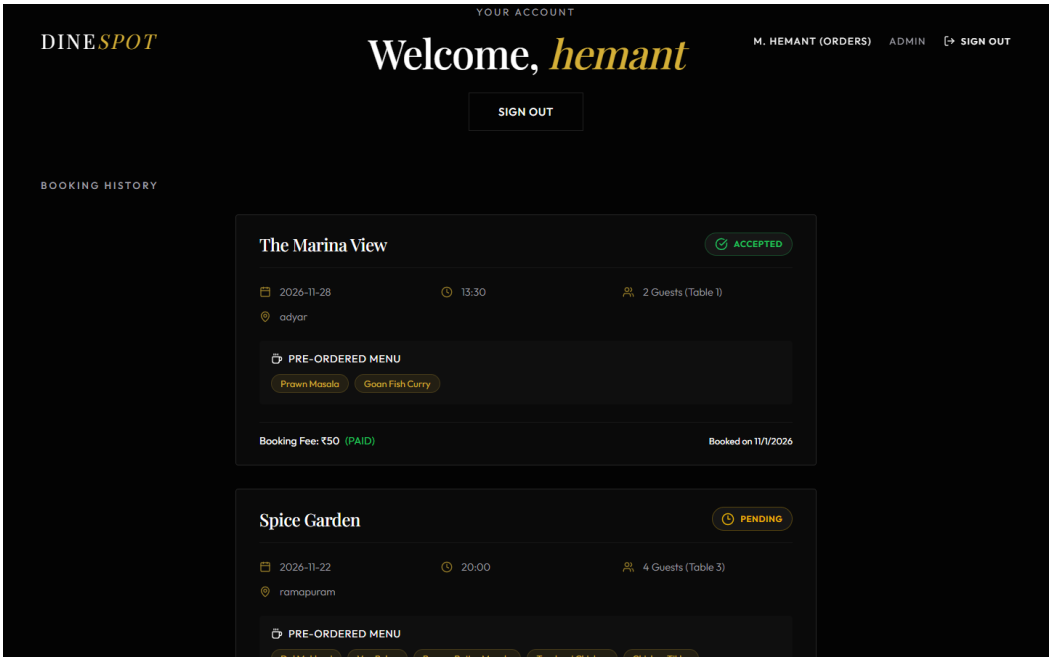


Figure 5.11 Booking History on the Profile Page

Figure 5.12 shows the admin dashboard with two pending requests, each offering Accept and Reject actions.

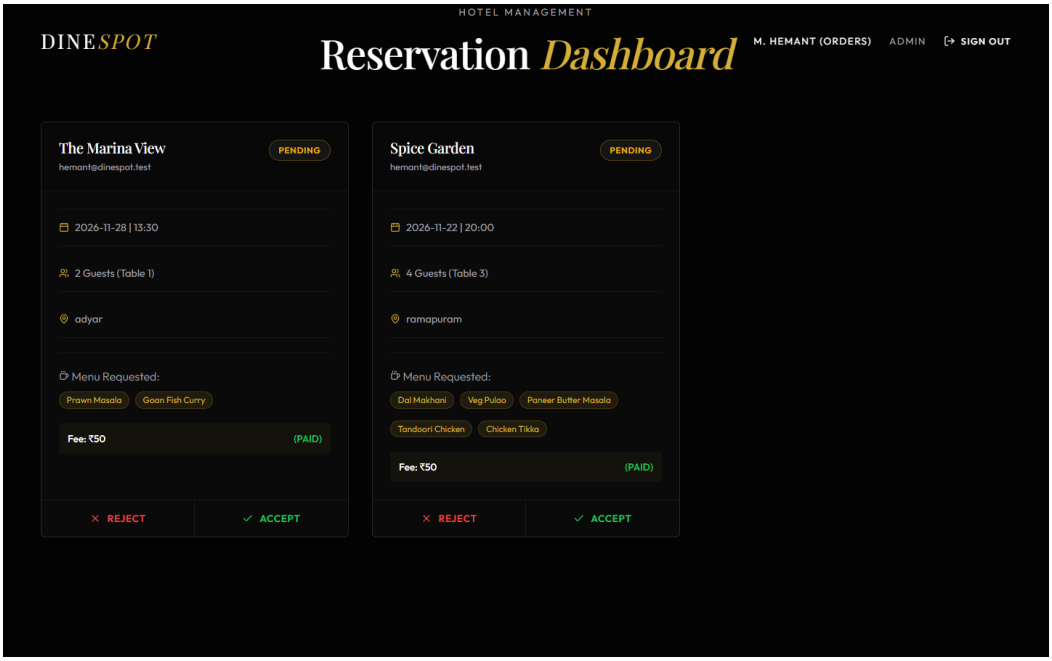


Figure 5.12 Admin Reservation Dashboard

After Accept is clicked, the badge changes to ACCEPTED and the Mark Completed action appears (Figure 5.13).

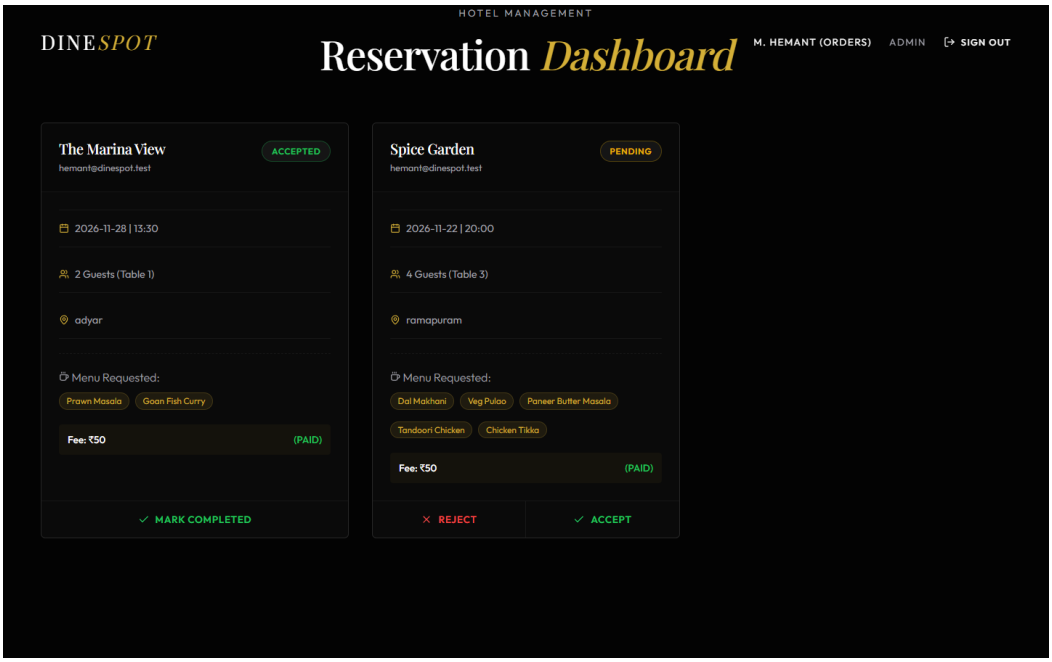


Figure 5.13 Dashboard After Accepting a Booking

5.2 TESTING AND VALIDATION

Testing combined API tests and interface tests. API tests were run with curl against the running backend, and each response body and HTTP status code was recorded. Interface tests were performed in Google Chrome by driving the real pages. The frontend production build and the linter were also run. A test is marked **Pass** when the actual result matched the expected result, **Fail** when it did not, and **Not verified** when it was not executed. Table 5.1 lists the test cases.

Table 5.1 Test cases and results

ID	Feature tested	Input or action	Expected result	Actual result	Status
TC01	List restaurants	GET /api/restaurants	All restaurants, highest rating first	17 returned; first “The Grand Table” (4.8); order verified	Pass
TC02	Search by area	GET ?q=adyar	Only Adyar restaurants	The Marina View, Copper Chimney	Pass
TC03	Search by region param	GET ?region=anna	Anna Nagar restaurants	3 Anna Nagar restaurants	Pass
TC04	Area with no data	Click Koyambedu chip	Empty-result message	[] returned; message shown (Fig. 5.4)	Pass
TC05	Search by cuisine	GET ?q=Seafood	Seafood restaurants (search bar hint says “neighborhood or cuisine”)	0 results; API matches only area and name	Fail
TC06	Restaurant by id	GET /api/restaurants/ram001	Spice Garden, 200	Spice Garden, 200	Pass
TC07	Unknown restaurant id	GET /api/restaurants/xyz999	404 Restaurant not found	404 Restaurant not found	Pass
TC08	Valid booking	POST, ram001, 4 guests, 19:30	Booking created on smallest fitting table	200, Table 3 (capacity 4)	Pass
TC09	Missing e-mail	POST without userEmail	400 userEmail is required	400 userEmail is required	Pass
TC10	Wrong date format	date = 15-11-2026	400 invalid date format	400 Invalid date format. Use YYYY-MM-DD	Pass
TC11	Guest limit	people = 25	400 guests out of range	400 guests must be between 1 and 20	Pass
TC12	Unknown restaurant in booking	restaurantId = nope	404	404 Restaurant not found	Pass
TC13	Same slot, second party	Another 4-guest booking at same slot	Next free table, no double booking	200, Table 4 (capacity 4)	Pass
TC14	Party larger than any table	8 guests at ram001 (largest table 6)	409 no table available	409 No table available…	Pass

ID	Feature tested	Input or action	Expected result	Actual result	Status
TC15	Slot exhaustion	11 bookings of 2 guests, ram003 (10 tables), same slot	10 succeed, 11th rejected	Tables 1, 2, 8, 9 (2-seat) then 4- and 6-seat; 11th → 409	Pass
TC16	Different slot	Same restaurant, 14:00	Tables free again	200, Table 1	Pass
TC17	Past date	date = 2020-01-01	Request rejected	200, booking created	Fail
TC18	Outside opening hours	03:00 at restaurant opening 12:00 PM	Request rejected	200, booking created	Fail
TC19	Booking history	GET /api/orders?email=…	Only that user's bookings, menu as array	Correct rows; menu_items parsed	Pass
TC20	History without e-mail	GET /api/orders	400	400 email query param required	Pass
TC21	Accept booking	PUT /api/orders/4/status accepted	200; status stored	200; history shows accepted	Pass
TC22	Unknown booking	PUT /api/orders/999/status	404	404 Booking not found	Pass
TC23	Missing status	PUT with empty body	400	400 status is required	Pass
TC24	Invalid status value	PUT status = xyz	400 rejected	200 “updated to xyz”	Fail
TC25	Unknown route	GET /api/unknown	404 JSON	404 Route not found	Pass
TC26	Sign in (UI)	Enter e-mail and password	Navbar shows user, Admin, Sign Out	Shown as “M. hemant (Orders)”	Pass
TC27	Password check (UI)	Any non-empty password	Wrong password rejected	Accepted; no server-side check	Fail
TC28	Empty menu (UI)	Submit with no dish selected	Error message, no request	Message shown (Fig. 5.8)	Pass
TC29	Complete reservation (UI)	Fill form, select 5 dishes, confirm	Confirmation summary	Shown (Fig. 5.10)	Pass
TC30	Admin actions (UI)	Click Accept on a card	Badge ACCEPTED, Mark Completed shown	As expected (Fig. 5.13)	Pass
TC31	Profile via Navbar (UI)	Click user name link	Booking history	Shown (Fig. 5.11)	Pass
TC32	Profile reload (UI)	Open /profile directly while signed in	Booking history	Redirected to /login	Fail
TC33	Production build	vite build	Build succeeds	Built; JS bundle 424 kB (135 kB gzip)	Pass
TC34	Static analysis	oxlint	No errors	0 errors, 11 warnings (unused imports and variables)	Pass
TC35	Concurrent requests	Simultaneous bookings from several clients	No double booking	Not executed	Not verified
TC36	Mobile layout	Narrow screen widths	Usable layout	Not executed	Not verified

ID	Feature tested	Input or action	Expected result	Actual result	Status
TC37	Vercel deployment	Deploy using vercel.json	Application reachable online	Not executed	Not verified

The results are summarised in Table 5.2.

Table 5.2 Summary of test results

Category	Executed	Pass	Fail	Not verified
Restaurant API (TC01–TC07)	7	6	1	0
Booking API (TC08–TC18)	11	9	2	0
History and admin API (TC19–TC25)	7	6	1	0
User interface (TC26–TC32)	7	5	2	0
Build and lint (TC33–TC34)	2	2	0	0
Not executed (TC35–TC37)	0	0	0	3
Total	34	28	6	3

5.3 RESULTS AND DISCUSSION

The tests show that the core reservation workflow works as designed. Restaurants are listed and searched correctly, every validation rule written in the booking handler returns the intended message and status code, and the transaction prevents a table from being given twice for the same slot. The slot-exhaustion test (TC15) is the clearest evidence of the allocation strategy. The first four parties of two received the four two-seat tables, the next four received four-seat tables and the last two received six-seat tables. Once all ten tables were taken, the eleventh request was refused with HTTP 409, while a request for a different time at the same restaurant immediately received Table 1 again (TC16).

The interface tests confirm that a customer can sign in, choose a restaurant, select dishes, receive a confirmation and see the booking on the Profile page, and that the administrator's Accept action changes the stored status and the available actions. The frontend builds for production without errors, and the linter reports only warnings for unused imports and variables.

The six failed tests point to specific gaps rather than broken features. The server does not reject past dates, times outside opening hours, or status values other than the four used by the interface (TC17, TC18, TC24). The search bar suggests that cuisine can be searched, but the API matches only area and name (TC05). Sign-in is a client-side session that does not check

the password (TC27), and the admin dashboard is reachable by anyone who opens `/admin`. Finally, a signed-in user who reloads the Profile page is redirected to sign in because the session is restored after the first render (TC32). In addition, the ₹50 fee is recorded as paid without a payment gateway, the `available_tables` and `available_seats` columns hold seed values that are not updated by bookings, and the Leaflet map component is not mounted. These points are addressed in the future enhancements in Chapter 6.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

DineSpot demonstrates a complete full-stack workflow for restaurant discovery and table reservation built with React, Express and SQLite. Customers can browse a rated collection of Chennai restaurants, narrow it by area, reserve a table for a specific date, time and party size, pre-order dishes and follow the status of their bookings. Hotel staff can review every request on a single dashboard and accept, reject or complete it.

The most important technical outcome is the transactional smallest-fit table allocation. By selecting the least-capacity free table inside a single SQLite transaction, the system both prevents double booking for a slot and keeps larger tables free for larger groups, as confirmed by the slot-exhaustion test. The choice of SQLite with `better-sqlite3` keeps the project self-contained: the schema and seed data are created automatically and no separate database server is needed.

Testing also identified clear limits of the current version: authentication is client-side, the admin dashboard is unprotected, payment is simulated, and some server-side validations are missing. These are documented honestly so that the next iteration can address them. Within its stated scope, DineSpot meets its objectives and provides a sound base for a production-grade reservation service.

6.2 FUTURE ENHANCEMENTS

- **Server-side authentication:** store users in the existing `users` table with bcrypt-hashed passwords, issue JSON Web Tokens on sign-in, and require a valid token for booking and history requests.
- **Role-based admin access:** add an admin role and protect `/api/orders/admin` and the status endpoint so that only staff can use them.
- **Stricter validation:** reject past dates and times outside opening hours, and accept only the defined status values.
- **Payment gateway:** integrate an online payment service so that the booking fee is marked paid only after a confirmed transaction.

- **Cancellation and time windows:** let customers cancel bookings (using the cancelled status already respected by the allocation query) and treat a booking as occupying a table for a duration instead of an exact time.
- **Live availability:** compute available tables and seats from bookings instead of seed values, and show them on the cards.
- **Map view and cuisine search:** mount the existing Leaflet MapComponent on the Home page and extend the search query to the cuisine column.
- **Notifications:** send e-mail or SMS updates when a booking is accepted, rejected or completed.
- **Hosted database:** move to a server database or managed SQLite service so that bookings persist in a serverless deployment.

Table 6.1 groups these enhancements into a phased roadmap, starting with the security and validation changes that a public deployment would need first.

Table 6.1 Roadmap of future work

Phase	Enhancement	Benefit
1	Server-side authentication, admin role, strict validation	Security and data correctness
2	Payment gateway, cancellation, live availability	Real-world booking operations
3	Map view, cuisine search, notifications	Better discovery and communication
4	Hosted database and cloud deployment	Availability to real users

REFERENCES / BIBLIOGRAPHY

[1] Meta Open Source, “React Documentation,” react.dev. [Online]. Available: https://react.dev/

[2] Vite Contributors, “Vite Guide,” vite.dev. [Online]. Available: https://vite.dev/guide/

[3] Remix Software, “React Router Documentation,” reactrouter.com. [Online]. Available: https://reactrouter.com/

[4] OpenJS Foundation, “Node.js Documentation,” nodejs.org. [Online]. Available: https://nodejs.org/docs/latest/api/

[5] OpenJS Foundation, “Express – Node.js Web Application Framework,” expressjs.com. [Online]. Available: https://expressjs.com/

[6] SQLite Consortium, “SQLite Documentation,” sqlite.org. [Online]. Available: https://www.sqlite.org/docs.html

[7] SQLite Consortium, “Write-Ahead Logging,” sqlite.org. [Online]. Available: https://www.sqlite.org/wal.html

[8] SQLite Consortium, “SQLite Foreign Key Support,” sqlite.org. [Online]. Available: https://www.sqlite.org/foreignkeys.html

[9] WiseLibs, “better-sqlite3,” GitHub repository. [Online]. Available: https://github.com/WiseLibs/better-sqlite3

[10] Express.js, “cors – Node.js CORS middleware,” GitHub repository. [Online]. Available: https://github.com/expressjs/cors

[11] Motion, “Motion for React (Framer Motion) Documentation,” motion.dev. [Online]. Available: https://motion.dev/

[12] Lucide Contributors, “Lucide Icons,” lucide.dev. [Online]. Available: https://lucide.dev/

[13] V. Agafonkin, “Leaflet – an open-source JavaScript library for interactive maps,” leafletjs.com. [Online]. Available: https://leafletjs.com/

[14] Mozilla, “Fetch API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[15] Mozilla, “Web Storage API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[16] Vercel, “Vercel Documentation,” vercel.com. [Online]. Available: https://vercel.com/docs

[17] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[18] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” Dec. 2017.

[19] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[20] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

APPENDICES (SOURCE CODE)

This appendix reproduces the main backend source files and the core frontend files of DineSpot. Styling files, the remaining seed records and the page components already excerpted in Chapter 4 are omitted to keep the report within the page limit. The project contains no secret keys or credentials.

Listing A.1 ***Project structure***

```
restaurant_prebooking/
|-- vercel.json
|-- backend/
|   |-- package.json
|   |-- server.js
|   |-- data.js
|   |-- database/
|   |   |-- database.js
|   |   `-- dinespot.db
|   |-- routes/
|   |   |-- restaurants.js
|   |   `-- bookings.js
|   `-- seed/
|       `-- seedRestaurants.js
|-- frontend/
|   |-- index.html
|   |-- package.json
|   |-- vite.config.js
|   `-- src/
|       |-- main.jsx
|       |-- App.jsx
|       |-- index.css, App.css
|       |-- context/AuthContext.jsx
|       |-- components/  Navbar, Hero, RestaurantCard, MapComponent (.jsx/.css)
|       `-- pages/       Home, Login, OrderSummary, Profile, Admin (.jsx/.css)
`-- legacy/              earlier static HTML/CSS/JS prototype
```

Listing A.2 ***Frontend development proxy (frontend/vite.config.js)***

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'


// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true,
      },
    },
  },
})
```

Listing A.3 ***Express server (backend/server.js)***

```
const express = require('express');
const cors = require('cors');
const seedRestaurants =
  require('./seed/seedRestaurants.js');


const restaurantRoutes = require('./routes/restaurants');
```

```
const bookingRoutes = require('./routes/bookings');

const app = express();

// ── Middleware ──────────────────────────────────────────────────────────
app.use(cors());
app.use(express.json());

// ── Request logger (dev) ────────────────────────────────────────────────
app.use((req, _res, next) => {
  console.log(`${new Date().toISOString()}  ${req.method} ${req.url}`);
  next();
});

// ── Seed database on startup ────────────────────────────────────────────
seedRestaurants();

// ── Routes ──────────────────────────────────────────────────────────────
app.use('/api/restaurants', restaurantRoutes);
app.use('/api/orders', bookingRoutes);   // backward compat with frontend
app.use('/api/bookings', bookingRoutes);   // REST alias

// ── 404 handler ─────────────────────────────────────────────────────────
app.use((_req, res) => {
  res.status(404).json({ success: false, message: 'Route not found' });
});

// ── Global error handler ────────────────────────────────────────────────
// eslint-disable-next-line no-unused-vars
app.use((err, _req, res, _next) => {
  console.error('Unhandled error:', err);
  res.status(500).json({ success: false, message: 'Internal server error' });
});

// ── Start ───────────────────────────────────────────────────────────────
const PORT = process.env.PORT || 5000;
if (process.env.NODE_ENV !== 'test' && !process.env.VERCEL) {
  app.listen(PORT, () => {
    console.log(`🚀 DineSpot backend running on http://localhost:${PORT}`);
  });
}

module.exports = app;
```

Listing A.4 ***Database connection and schema (backend/database/database.js)***

```
const Database = require('better-sqlite3');
const path = require('path');
const fs = require('fs');

// Ensure database directory exists
const dbDir = path.join(__dirname);
if (!fs.existsSync(dbDir)) fs.mkdirSync(dbDir, { recursive: true });

const db = new Database(path.join(dbDir, 'dinespot.db'));

// Enable WAL mode for better concurrent read performance
db.pragma('journal_mode = WAL');
// Enforce foreign keys
db.pragma('foreign_keys = ON');

// ─── Schema ──────────────────────────────────────────────────────────────

db.exec(`
  CREATE TABLE IF NOT EXISTS users (
```

```
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name       TEXT NOT NULL,
    email      TEXT UNIQUE NOT NULL,
    created_at TEXT DEFAULT (datetime('now'))
  );

  CREATE TABLE IF NOT EXISTS restaurants (
    id               TEXT PRIMARY KEY,
    name             TEXT NOT NULL,
    region           TEXT NOT NULL,
    address          TEXT,
    cuisine          TEXT,
    rating           REAL DEFAULT 0,
    total_tables     INTEGER DEFAULT 0,
    available_tables INTEGER DEFAULT 0,
    total_seats      INTEGER DEFAULT 0,
    available_seats  INTEGER DEFAULT 0,
    image_url        TEXT,
    price_range      TEXT,
    tags             TEXT,
    open_time        TEXT,
    close_time       TEXT,
    phone            TEXT,
    lat              REAL,
    lng              REAL,
    created_at       TEXT DEFAULT (datetime('now'))
  );

  CREATE TABLE IF NOT EXISTS restaurant_tables (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    restaurant_id TEXT NOT NULL REFERENCES restaurants(id) ON DELETE CASCADE,
    table_number  INTEGER NOT NULL,
    capacity      INTEGER NOT NULL,
    status        TEXT DEFAULT 'available'
  );

  CREATE TABLE IF NOT EXISTS bookings (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_email    TEXT NOT NULL,
    restaurant_id TEXT NOT NULL REFERENCES restaurants(id),
    table_id      INTEGER REFERENCES restaurant_tables(id),
    booking_date  TEXT NOT NULL,
    booking_time  TEXT NOT NULL,
    guests        INTEGER NOT NULL,
    menu_items    TEXT,
    location      TEXT,
    fee_amount    REAL DEFAULT 50.0,
    payment_status TEXT DEFAULT 'pending',
    status        TEXT DEFAULT 'pending',
    created_at    TEXT DEFAULT (datetime('now'))
  );
`);

module.exports = db;
```

Listing A.5 ***Seeding (backend/seed/seedRestaurants.js)***

```
// seed/seedRestaurants.js
// Idempotent seed – skips restaurants that already exist in SQLite.

const db = require('../database/database');
const { RESTAURANTS_DB } = require('../data');

function seedRestaurants() {
  const insertRestaurant = db.prepare(`
    INSERT OR IGNORE INTO restaurants
      (id, name, region, address, cuisine, rating, total_tables, available_tables,
       total_seats, available_seats, image_url, price_range, tags,
       open_time, close_time, phone, lat, lng)
    VALUES
```

```
      (@id, @name, @region, @address, @cuisine, @rating, @totalTables, @availableTables,
       @totalSeats, @availableSeats, @image, @priceRange, @tags,
       @openTime, @closeTime, @phone, @lat, @lng)
  `);

  const insertTable = db.prepare(`
    INSERT OR IGNORE INTO restaurant_tables (restaurant_id, table_number, capacity)
    SELECT @restaurantId, @tableNumber, @capacity
    WHERE NOT EXISTS (
      SELECT 1 FROM restaurant_tables
      WHERE restaurant_id = @restaurantId AND table_number = @tableNumber
    )
  `);

  const seedAll = db.transaction(() => {
    let seeded = 0;
    for (const r of RESTAURANTS_DB) {
      const result = insertRestaurant.run({
        id: r.id,
        name: r.name,
        region: r.region,
        address: r.address,
        cuisine: r.cuisine,
        rating: r.rating,
        totalTables: r.totalTables,
        availableTables: r.availableTables,
        totalSeats: r.totalSeats,
        availableSeats: r.availableSeats,
        image: r.image,
        priceRange: r.priceRange,
        tags: JSON.stringify(r.tags),
        openTime: r.openTime,
        closeTime: r.closeTime,
        phone: r.phone,
        lat: r.lat,
        lng: r.lng
      });

      if (result.changes > 0) {
        seeded++;
        // Generate sensible tables for each restaurant
        const tableSizes = [2, 2, 4, 4, 4, 6, 6];
        for (let i = 0; i < r.totalTables; i++) {
          insertTable.run({
            restaurantId: r.id,
            tableNumber: i + 1,
            capacity: tableSizes[i % tableSizes.length]
          });
        }
      }
    }
    return seeded;
  });

  const seeded = seedAll();
  if (seeded > 0) {
    console.log(`✅ Seeded ${seeded} restaurants into SQLite`);
  } else {
    console.log('✅ Restaurant data already present - skipped seed');
  }
}

module.exports = seedRestaurants;
```

Listing A.6 ***First seed record (backend/data.js)***

```
// restaurants-data.js
// Mock database of restaurants across Chennai regions (with GPS coordinates)
```

```
const RESTAURANTS_DB = [
  // ==================== RAMAPURAM ====================
  {
    id: "ram001",
    name: "Spice Garden",
    region: "ramapuram",
    address: "14, Mount Poonamallee Rd, Ramapuram",
    cuisine: "South Indian & North Indian",
    rating: 4.5,
    totalTables: 12,
    availableTables: 7,
    totalSeats: 60,
    availableSeats: 35,
    image: "https://images.unsplash.com/photo-1517248135467-4c7edcad34c4?w=400&q=80",
    priceRange: "₹₹",
    tags: ["Family Friendly", "AC", "Parking"],
    openTime: "10:00 AM",
    closeTime: "10:00 PM",
    phone: "+91 98410 11223",
    lat: 13.0399,
    lng: 80.1760
  },
  {
  // ... 16 more restaurant records
];
```

Listing A.7 ***Restaurant routes (backend/routes/restaurants.js)***

```
// routes/restaurants.js
const express = require('express');
const router = express.Router();
const db = require('../database/database');


// Helper: parse tags JSON safely
const parseRestaurant = (r) => ({
  ...r,
  tags: (() => { try { return JSON.parse(r.tags || '[]'); } catch { return []; } })(),
  availableTables: r.available_tables,
  totalTables: r.total_tables,
  availableSeats: r.available_seats,
  totalSeats: r.total_seats,
  image: r.image_url,
  priceRange: r.price_range,
  openTime: r.open_time,
  closeTime: r.close_time,
});

// GET /api/restaurants  (optionally filter by ?region=...)
router.get('/', (req, res) => {
  try {
    const { region, q } = req.query;
    let stmt, rows;


    if (region) {
      stmt = db.prepare(`SELECT * FROM restaurants WHERE LOWER(region) LIKE LOWER(?)  ORDER
    BY rating DESC`);
      rows = stmt.all(`%${region}%`);
    } else if (q) {
      stmt = db.prepare(`SELECT * FROM restaurants WHERE LOWER(region) LIKE LOWER(?) OR
    LOWER(name) LIKE LOWER(?) ORDER BY rating DESC`);
      rows = stmt.all(`%${q}%`, `%${q}%`);
    } else {
      rows = db.prepare('SELECT * FROM restaurants ORDER BY rating DESC').all();
    }


    res.json(rows.map(parseRestaurant));
  } catch (err) {
    console.error('GET /restaurants error:', err);
    res.status(500).json({ success: false, message: 'Failed to fetch restaurants' });
  }
});
```

```
// GET /api/restaurants/:id
router.get('/:id', (req, res) => {
  try {
    const row = db.prepare('SELECT * FROM restaurants WHERE id = ?').get(req.params.id);
    if (!row) return res.status(404).json({ success: false, message: 'Restaurant not found'
    });
    res.json(parseRestaurant(row));
  } catch (err) {
    console.error('GET /restaurants/:id error:', err);
    res.status(500).json({ success: false, message: 'Failed to fetch restaurant' });
  }
});

module.exports = router;
```

Listing A.8 ***Booking routes (backend/routes/bookings.js)***

```
// routes/bookings.js
const express = require('express');
const router = express.Router();
const db = require('../database/database');

// POST /api/orders
// Body: { userEmail, orderDetails: { restaurantId, date, time, people, menuItems } }
router.post('/', (req, res) => {
  const { userEmail, orderDetails } = req.body;

  // ── Validation ──────────────────────────────────────────────────
  if (!userEmail) return res.status(400).json({ success: false, message: 'userEmail is
    required' });
  if (!orderDetails) return res.status(400).json({ success: false, message: 'orderDetails
    is required' });

  const { restaurantId, date, time, people, menuItems } = orderDetails;

  if (!restaurantId) return res.status(400).json({ success: false, message: 'restaurantId
    is required' });
  if (!date) return res.status(400).json({ success: false, message: 'date is required' });
  if (!time) return res.status(400).json({ success: false, message: 'time is required' });

  const guests = parseInt(people, 10);
  if (isNaN(guests) || guests < 1 || guests > 20) {
    return res.status(400).json({ success: false, message: 'guests must be between 1 and
    20' });
  }

  if (!/^\d{4}-\d{2}-\d{2}$/.test(date)) {
    return res.status(400).json({ success: false, message: 'Invalid date format. Use YYYY-
    MM-DD' });
  }

  // ── Check restaurant exists ─────────────────────────────────────
  const restaurant = db.prepare('SELECT * FROM restaurants WHERE id =
    ?').get(restaurantId);
  if (!restaurant) {
    return res.status(404).json({ success: false, message: 'Restaurant not found' });
  }

  // ── Find best available table and book atomically ───────────────
  const bookingTx = db.transaction(() => {
    // IDs of tables already booked for this slot
    const bookedRows = db.prepare(`
      SELECT table_id FROM bookings
      WHERE restaurant_id = ? AND booking_date = ? AND booking_time = ?
        AND status != 'cancelled'
    `).all(restaurantId, date, time);
```

```
    const bookedIds = bookedRows.map(b => b.table_id);

    // Build query depending on whether any tables are already booked
    let availableTable;
    if (bookedIds.length === 0) {
      // No bookings yet - all tables are free
      availableTable = db.prepare(`
        SELECT * FROM restaurant_tables
        WHERE restaurant_id = ? AND capacity >= ?
        ORDER BY capacity ASC
        LIMIT 1
      `).get(restaurantId, guests);
    } else {
      // Exclude already-booked table IDs
      const placeholders = bookedIds.map(() => '?').join(',');
      availableTable = db.prepare(`
        SELECT * FROM restaurant_tables
        WHERE restaurant_id = ? AND capacity >= ?
          AND id NOT IN (${placeholders})
        ORDER BY capacity ASC
        LIMIT 1
      `).get(restaurantId, guests, ...bookedIds);
    }

    if (!availableTable) {
      return {
        success: false,
        status: 409,
        message: 'No table available for the selected time slot and guest count'
      };
    }

    // Insert the booking
    const result = db.prepare(`
      INSERT INTO bookings
        (user_email, restaurant_id, table_id, booking_date, booking_time, guests,
    menu_items, location, fee_amount, payment_status, status)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, 50.0, 'paid', 'pending')
    `).run(
      userEmail,
      restaurantId,
      availableTable.id,
      date,
      time,
      guests,
      JSON.stringify(menuItems || []),
      restaurant.region
    );

    return {
      success: true,
      bookingId: result.lastInsertRowid,
      tableNumber: availableTable.table_number,
      tableCapacity: availableTable.capacity,
      feeAmount: 50.0,
      paymentStatus: 'paid',
      message: `Table ${availableTable.table_number} booked successfully. Status is Pending
    Approval.`
    };
  });

  try {
    const result = bookingTx();
    return res.status(result.status || 200).json(result);
  } catch (err) {
    console.error('POST /orders error:', err);
    return res.status(500).json({ success: false, message: 'Unexpected error processing
    booking' });
  }
});
```

```
// GET /api/orders?email=...  - all bookings for a user
router.get('/', (req, res) => {
  const { email } = req.query;
  if (!email) return res.status(400).json({ success: false, message: 'email query param
    required' });

  try {
    const rows = db.prepare(`
      SELECT b.*,
             r.name       AS restaurant_name,
             r.address    AS restaurant_address,
             rt.table_number,
             rt.capacity
      FROM   bookings b
      JOIN   restaurants r  ON b.restaurant_id = r.id
      LEFT   JOIN restaurant_tables rt ON b.table_id = rt.id
      WHERE  b.user_email = ?
      ORDER  BY b.created_at DESC
    `).all(email);

    const parsed = rows.map(b => ({
      ...b,
      menu_items: (() => { try { return JSON.parse(b.menu_items || '[]'); } catch { return
    []; } })()
    }));

    res.json(parsed);
  } catch (err) {
    console.error('GET /orders error:', err);
    res.status(500).json({ success: false, message: 'Failed to fetch bookings' });
  }
});

// GET /api/orders/admin - fetch all bookings for admin
router.get('/admin', (req, res) => {
  try {
    const rows = db.prepare(`
      SELECT b.*,
             r.name       AS restaurant_name,
             r.address    AS restaurant_address,
             rt.table_number,
             rt.capacity
      FROM   bookings b
      JOIN   restaurants r  ON b.restaurant_id = r.id
      LEFT   JOIN restaurant_tables rt ON b.table_id = rt.id
      ORDER  BY b.created_at DESC
    `).all();

    const parsed = rows.map(b => ({
      ...b,
      menu_items: (() => { try { return JSON.parse(b.menu_items || '[]'); } catch { return
    []; } })()
    }));

    res.json(parsed);
  } catch (err) {
    console.error('GET /orders/admin error:', err);
    res.status(500).json({ success: false, message: 'Failed to fetch admin bookings' });
  }
});

// PUT /api/orders/:id/status - update booking status (admin)
router.put('/:id/status', (req, res) => {
  const { id } = req.params;
  const { status } = req.body;

  if (!status) return res.status(400).json({ success: false, message: 'status is required'
    });

  try {
```

```
    const result = db.prepare('UPDATE bookings SET status = ? WHERE id = ?').run(status,
    id);
    if (result.changes === 0) {
      return res.status(404).json({ success: false, message: 'Booking not found' });
    }
    res.json({ success: true, message: `Booking status updated to ${status}` });
  } catch (err) {
    console.error('PUT /orders/:id/status error:', err);
    res.status(500).json({ success: false, message: 'Failed to update status' });
  }
});

module.exports = router;
```

Listing A.9 ***Application routes (frontend/src/App.jsx)***

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Login from './pages/Login';
import OrderSummary from './pages/OrderSummary';

import Profile from './pages/Profile';
import Admin from './pages/Admin';

function App() {
  return (
    <div className="app-container">
      <Navbar />
      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/order-summary" element={<OrderSummary />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/admin" element={<Admin />} />
        </Routes>
      </main>
    </div>
  );
}

export default App;
```